

Lesson 4: Neural Networks



4.0 Lesson Overview



Lesson Overview

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



4.1 Building a Default Neural Network Model



Beyond Traditional Regression: Neural Networks



Copyright © SAS Institute Inc. All rights reserved.

Beyond Traditional Regression: Neural Networks

Neural Network



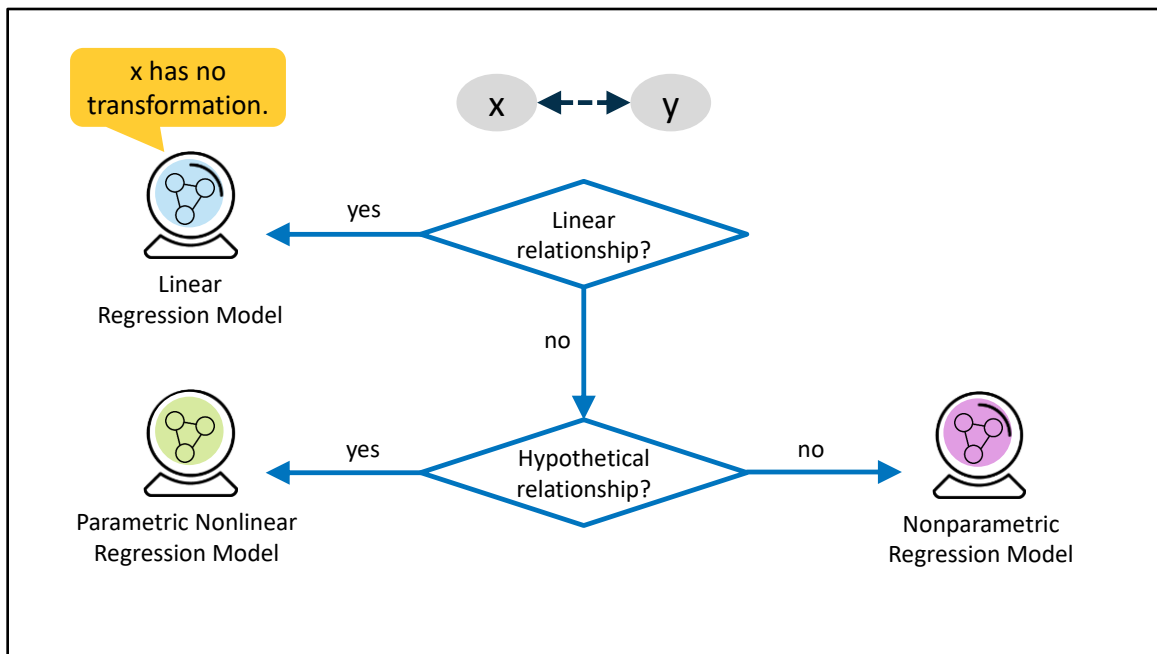
Traditional
Nonlinear Model



advantages

disadvantages

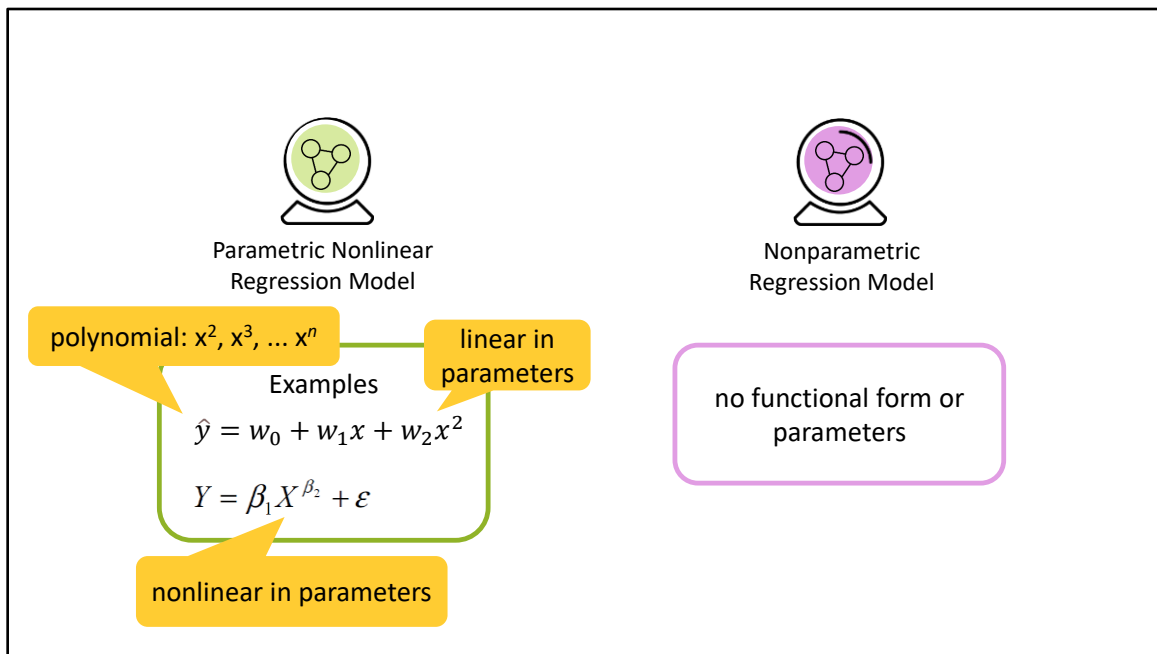
To understand the flexibility of neural networks, let's first look at the advantages and disadvantages of more traditional nonlinear models.



When an input variable x has a linear relationship with a target y , x appears in the model with no transformation.

If the relationship is nonlinear, but you can specify a hypothetical relationship between x and y , you can build a parametric nonlinear regression model.

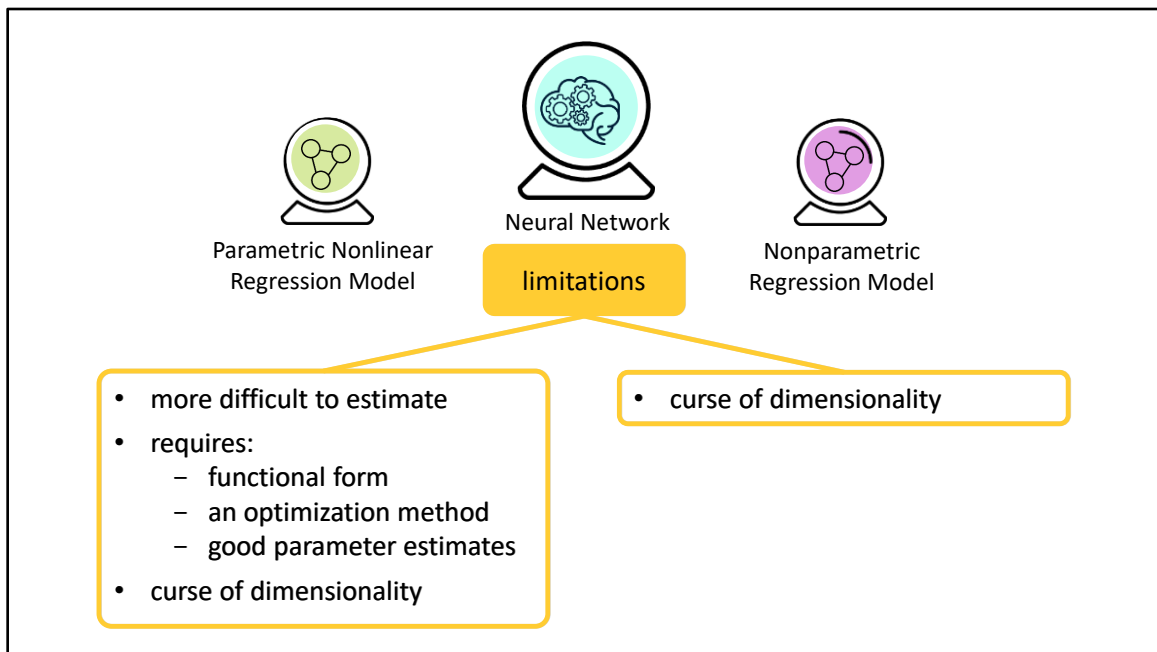
When it is not practical to specify the hypothetical relationship, you can build a nonparametric regression model.



A parametric nonlinear model requires you to specify the hypothetical relationship in a functional form. One approach is to add a transformation of x to the functional form. A popular choice is a polynomial. For a single input, x , polynomial degrees are x -squared (or quadratic), x -cubed (or cubed), and so on. For example, the model with a quadratic term that is shown here can model nonlinear relationships, but it is actually linear in parameters.

Below that is an example of a model that is nonlinear in parameters.

A nonparametric regression model has no functional form and, therefore, no parameters.

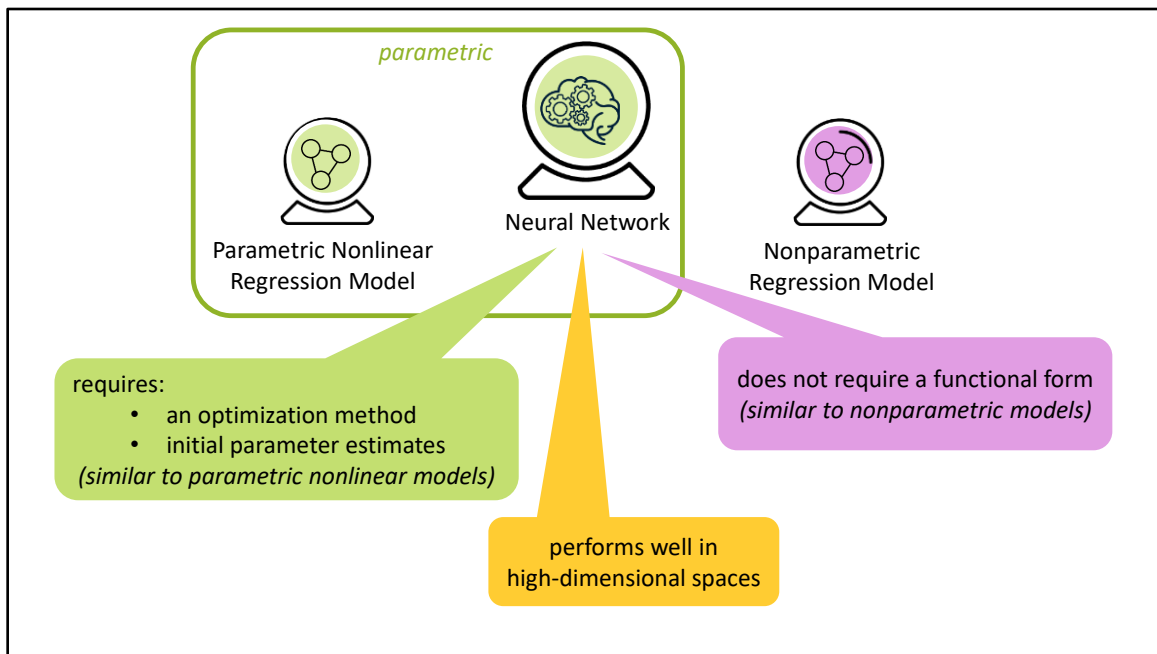


Traditional nonlinear regression models, which are parametric, and nonparametric regression models have several limitations.

Nonlinear regression models are more difficult to estimate than linear models. In addition to specifying the functional form, it is necessary to use an optimization method to efficiently guide the parameter search process. It is essential for the optimization process to start with good parameter estimates. Starting at a bad location in the parameter space results in an inferior model.

Traditional nonlinear modeling techniques become vastly more difficult as the number of inputs increases. Remember that this is called the curse of dimensionality. It is uncommon to see parametric nonlinear regression models with more than a few inputs, because deriving a suitable functional form becomes increasingly difficult as the number of inputs increases. Higher-dimension input spaces are also a challenge for nonparametric regression models.

Neural networks were developed to overcome these challenges.



Although neural networks are parametric nonlinear models, they are similar to nonparametric models in one way: neural networks do not require the functional form to be specified. This enables you to construct models when the relationship between the inputs and outputs is unknown.

Like other parametric nonlinear models, however, neural networks do require the use of an optimization process with initial parameter estimates.

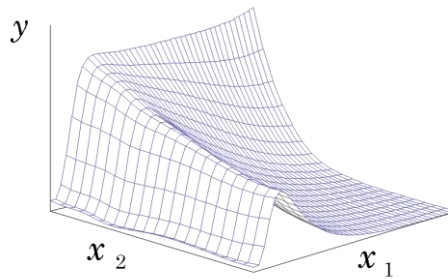
Also, unlike nonlinear parametric regression models and nonparametric regression models, neural networks generally perform well in sparse, high-dimensional spaces.



Neural Network

flexibility

Universal Approximator



A major benefit of neural networks is their unlimited flexibility.

A neural network is a universal approximator, which means that it can model any input-output relationship, no matter how complex. For example, this graph depicts the relationship between two inputs, x_1 and x_2 , and a target y . This complex relationship would be difficult to estimate accurately using traditional regression methods. However, a neural network can do the job.

Overcoming the Limitations of Neural Networks



Copyright © SAS Institute Inc. All rights reserved.

Overcoming the Limitations of Neural Networks

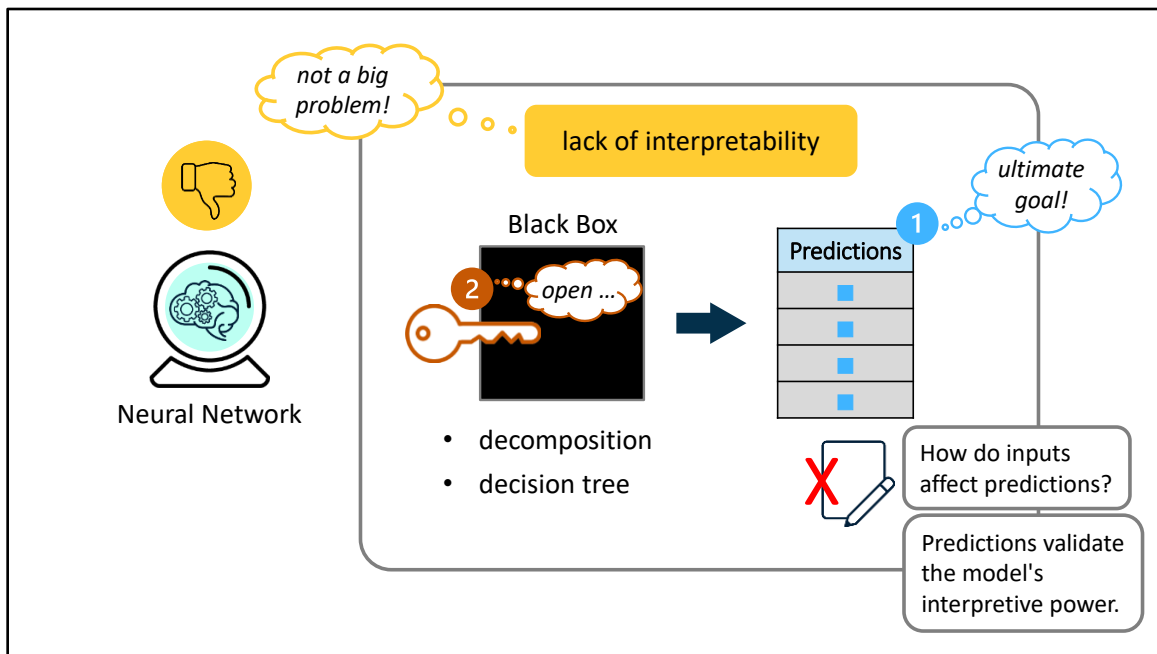


Neural Network

lack of interpretability

need for a strong signal

Neural networks overcome the main limitations of traditional regression methods, but they have a few limitations of their own: lack of interpretability and the need for a strong signal in the data.



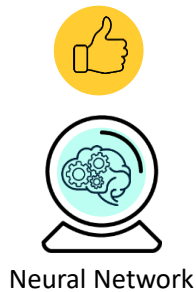
The lack of interpretability is the basis for the well-known *black box* objection to neural networks. Suggesting mystery, the term black box is often used simply to disparage neural networks by implying that they can never be interpreted.

However, the lack of interpretability is not as big a problem as it might appear. First, for many modeling tasks, pure prediction is the goal. Understanding how the inputs affect the prediction is of secondary importance. In other applications, the opposite is true: predictive power is a consideration only to the extent that it validates the interpretive power of the model. Neural networks are most appropriate for pure prediction tasks.

Second, there are methods that enable you to open the black box, at least partially (that is, to gain insight into how the neural network makes predictions).

One such method, called decomposition, approximates the parameters in a neural network using a set of IF-THEN rules.

A related approach is to use a decision tree to interpret the neural network's predictions.



lack of interpretability

Black Box

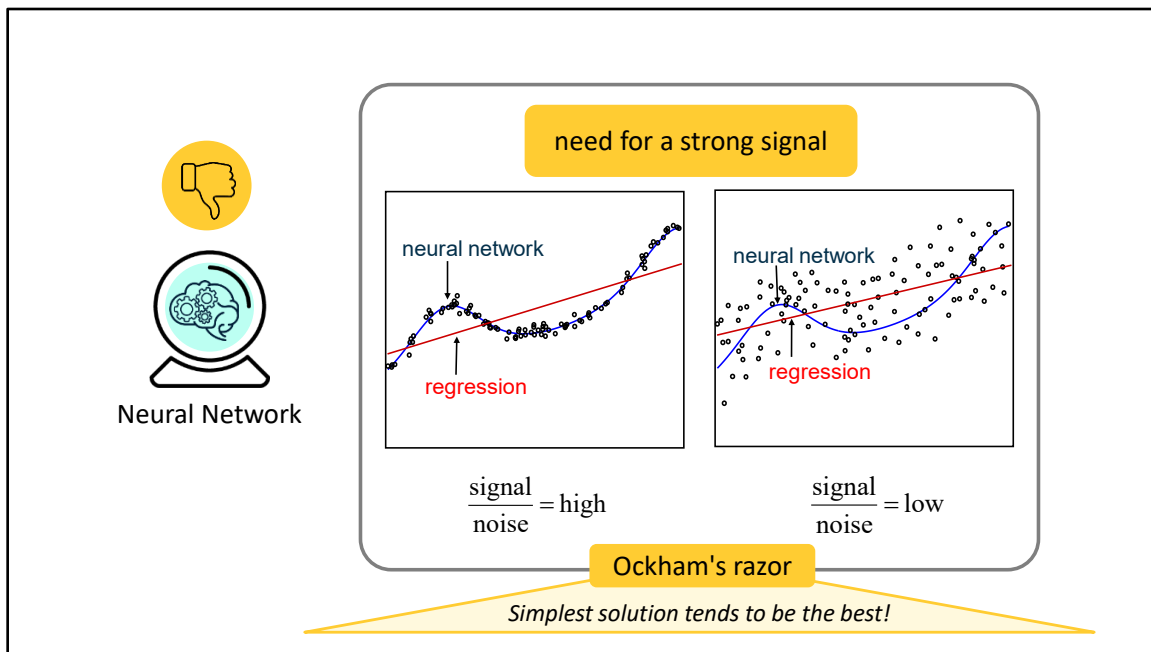


Model Interpretability

- Variable importance
- PD plots
- ICE plots
- LIME
- Hyper Shapley values

model neutral

To help you better understand your model, SAS Viya provides model interpretability techniques such as Variable Importance, PD plots, ICE plots, LIME, and Hyper Shapley values. These techniques are model agnostic, which means that they are applicable to any model that is generated by a supervised learning node. You can learn more about model interpretability later in the course.



Depending on the data, neural networks might not outperform simpler and easier to implement models like regression. In particular, neural networks require a high signal-to-noise ratio in the data. In the graph on the left, the neural network model fits the data well, as compared to the linear regression model. The data have a strong pattern (or signal) relative to the amount of variation (or noise) around the pattern. The signal-to-noise ratio is high.

The graph on the right tells a different story: the signal-to-noise ratio is low. The regression and neural network models offer a comparable fit to the data.

In a situation like this, Ockham's razor implies that there is no advantage to using the more complex neural network. What is Ockham's razor? It is a principle that has been around for centuries, and it states that the simplest solution tends to be the best.

No model is always best.



Neural Network



one of several tools in your analytics toolbox

In fact, no single type of model is always best, so think of neural networks as one of several tools in your analytics toolbox.

Question: Universal Approximators



Copyright © SAS Institute Inc. All rights reserved.

Question: Universal Approximators

Neural networks are universal approximators. This means that neural networks can do which of the following?

- a. model any input-output relationship, no matter how complex
- b. outperform any other type of model, no matter how strong the signal
- c. clearly indicate how the inputs affect the prediction, no matter the number of inputs

Answer: Universal Approximators

Neural networks are universal approximators. This means that neural networks can do which of the following?

- a. model any input-output relationship, no matter how complex
- b. outperform any other type of model, no matter how strong the signal
- c. clearly indicate how the inputs affect the prediction, no matter the number of inputs

CA: a

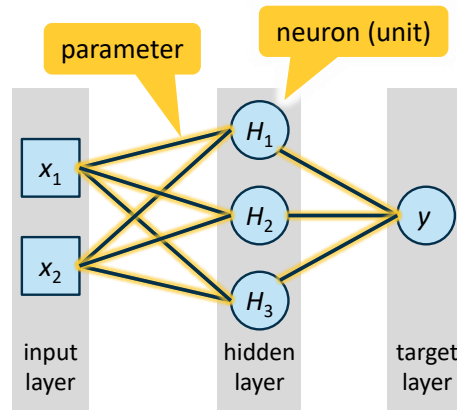
FB: Neural networks are called universal approximators because they can model any input-output relationship, no matter how complex.

Basics of Neural Networks

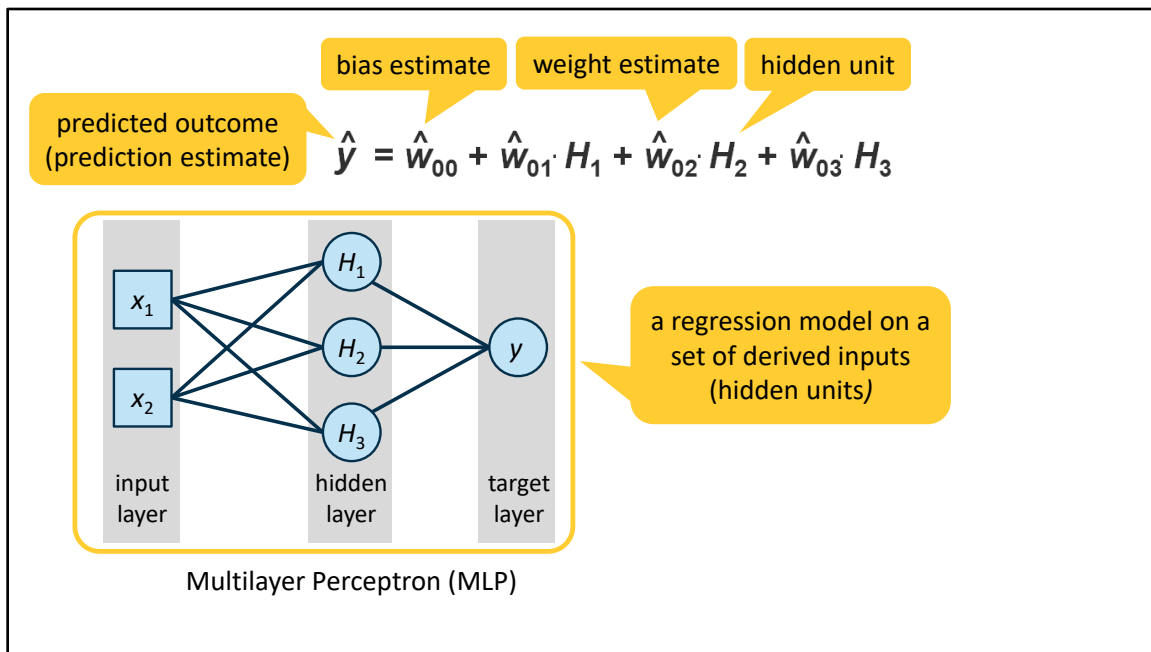


Copyright © SAS Institute Inc. All rights reserved.

Basics of Neural Networks



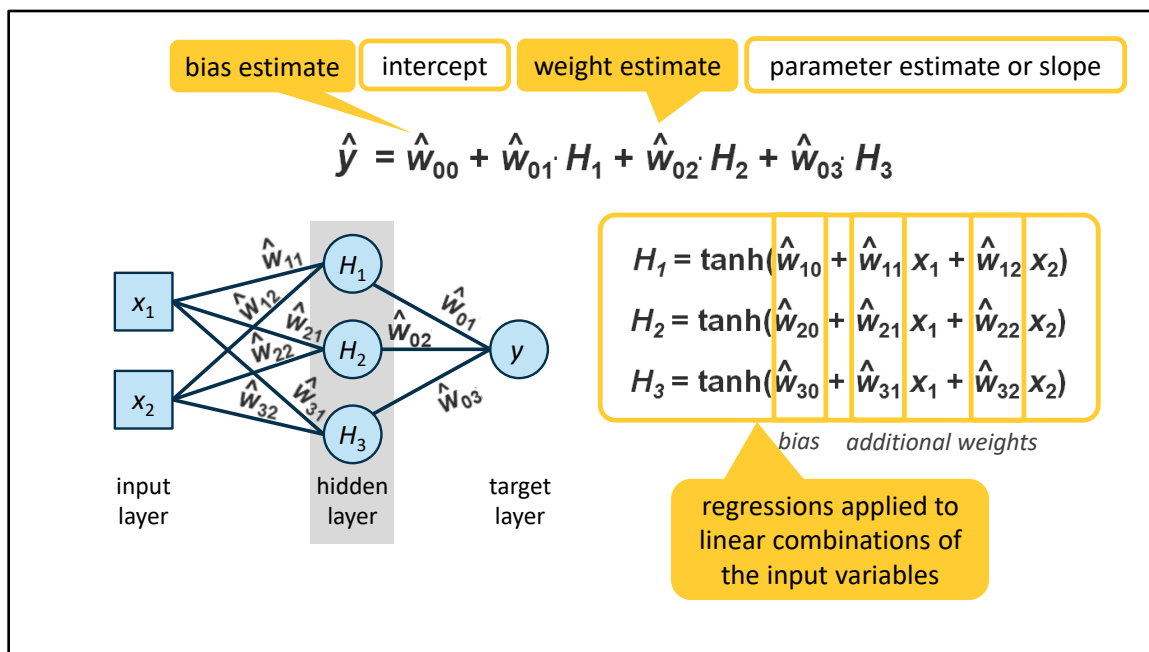
A neural network is structured in layers that contain neurons (or units). Lines represent the parameters that connect the neurons between layers.



This diagram shows a basic multilayer perceptron (or MLP), which is a common type of neural network. A basic MLP has three layers: an input layer, a hidden layer, and a target (or output) layer. In this particular MLP, the input layer has two inputs (x_1 and x_2) and the target layer has one target (y). The hidden layer has three hidden units (H_1 through H_3). So, what are hidden units?

A neural network can be thought of as a regression model on a set of derived inputs, called *hidden units*. The functional form (or the prediction formula) for this MLP is shown at the top.

In this example, the predicted outcome equals the sum of a bias estimate and three terms that are products of a weight estimate and a hidden unit.



Bias estimate is the neural network term for an intercept.

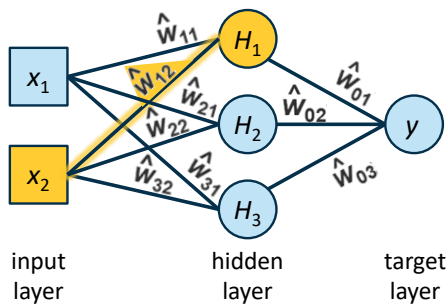
Weight estimate is the neural network term for a parameter estimate or slope.

When there is a single hidden layer, the hidden units can be thought of as regressions applied to linear combinations of the input variables. In this example, an equation for each of the three hidden units is shown on the right.

Each linear combination of the input variables contains additional weights and a bias term.

Remember that the lines in the neural network diagram represent the weights. The weights for the linear combinations of the input variables and the weights from the hidden units are now labeled in the diagram.

$$\hat{y} = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$



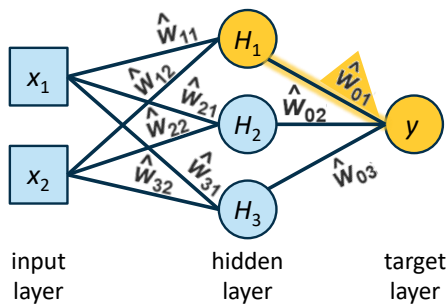
$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

For example, the line connecting hidden unit H_1 to input x_2 represents the weight $\hat{w}_{12}$.

$$\hat{y} = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$



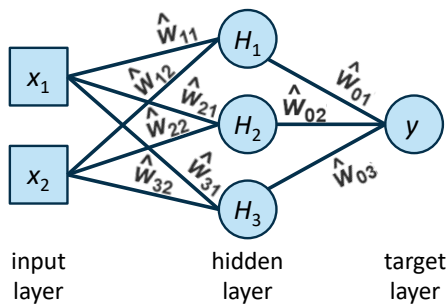
$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

And the line connecting the target to the hidden unit H_1 represents the weight w -hat sub zero one.

$$\hat{y} = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$



$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

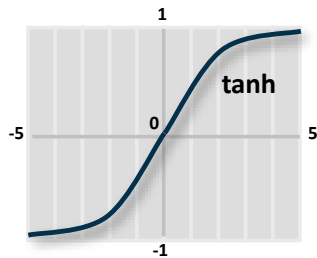
$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

bias terms

In each equation, the first w -hat term is the bias term, which is not represented in the diagram. The second subscript on a bias term is always zero.

$$\hat{y} = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$



$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

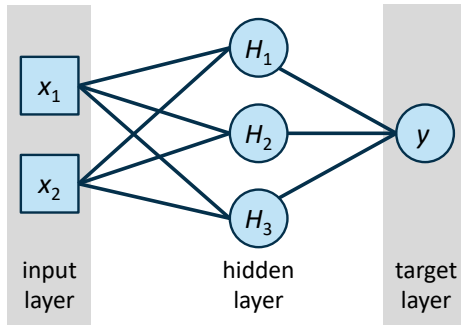
$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

activation function

The hidden unit regressions include an activation function. This activation function is a mathematical transformation that is applied to the input layer. In the equations shown here, the activation function is the hyperbolic tangent, which is the default in Model Studio. The hyperbolic tangent is a shift and rescaling of the logistic function.

can approximate virtually any association

$$\hat{y} = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$



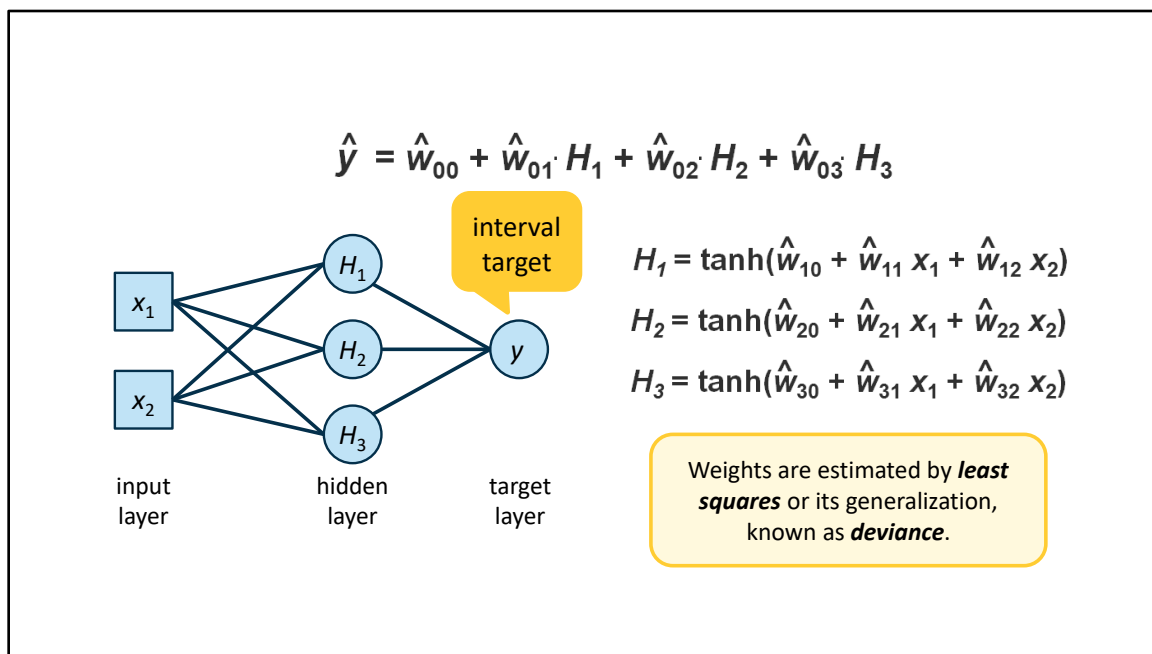
$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

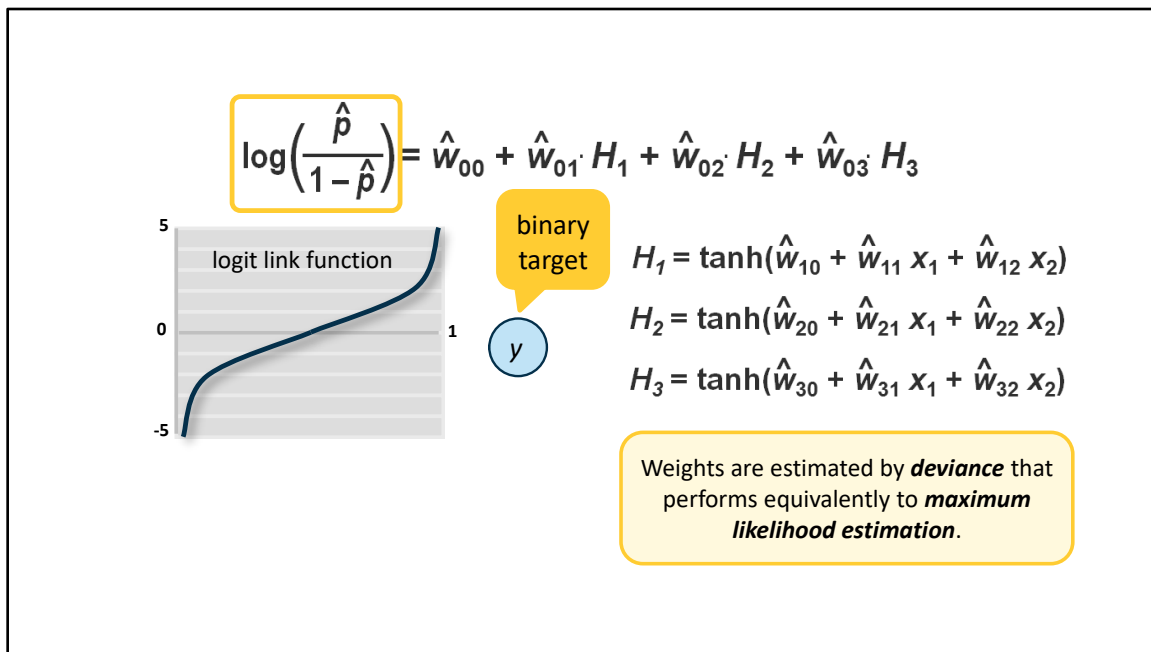
$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

specify the correct
number of hidden units

What makes neural networks interesting is their ability to approximate virtually any association between the inputs and the target. You simply specify the correct number of hidden units and find reasonable values for the weights. Specifying the correct number of hidden units involves some trial and error.



For interval targets, least squares estimation is usually used to estimate the weights. For certain distributions of interval targets, a generalization of least squares estimation, known as deviance estimation, is used instead.



When the target variable is binary, the main neural network regression equation uses the same logit link function as a logistic regression does. For binary targets, the weight estimation process uses deviance estimation. When the distribution is in the exponential family (as is the case with a binary target), deviance estimation performs equivalently to maximum likelihood estimation.

Question: Parameter Estimates



Copyright © SAS Institute Inc. All rights reserved.

Question: Parameter Estimates

Which of the following terms refers to a parameter estimate or slope that is associated with an input in a neural network?

- a. activation function
- b. bias estimate
- c. weight estimate
- d. hidden unit

Answer: Parameter Estimates

Which of the following terms refers to a parameter estimate or slope that is associated with an input in a neural network?

- a. activation function
- b. bias estimate
- ☒ c. weight estimate
- d. hidden unit

CA: c

FB: **Weight estimate** is the neural network term for a parameter estimate or slope.

Question: Intercept Estimate



Copyright © SAS Institute Inc. All rights reserved.

Question: Intercept Estimate

Which of the following terms refers to the intercept estimate in a neural network?

- a. activation function
- b. bias estimate
- c. weight estimate
- d. hidden unit

Answer: Intercept Estimate

Which of the following terms refers to the intercept estimate in a neural network?

- a. activation function
- ☒ b. bias estimate
- c. weight estimate
- d. hidden unit

CA: b

feedback: **Bias estimate** is the neural network term for an intercept.

Estimating Weights and Making Predictions



Copyright © SAS Institute Inc. All rights reserved.

Estimating Weights and Making Predictions

produces a ranking or logit score

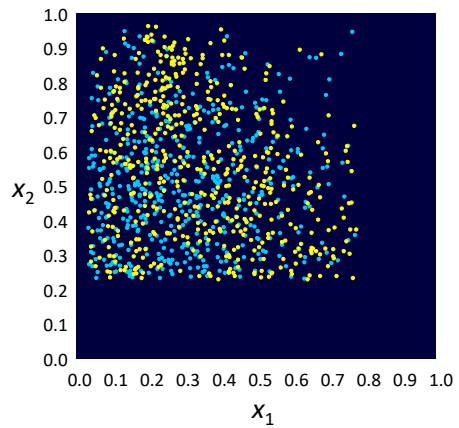
$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

binary
target

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$



To show how a neural network estimates weights and makes predictions, let's take another look at the two-color prediction problem. In this example, remember that the goal is to predict the target color based on the location in the plot. As with regressions, a neural network's predictions can be decisions, rankings, or estimates. In this example, the target is binary, so the estimated prediction formula produces a ranking or logit score.

1. Standardize (scale) the input variables.

$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

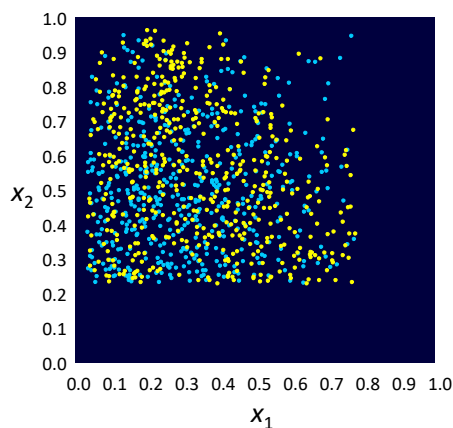
$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

Midrange

- midrange: 0
- minimum: -1
- maximum: 1

default in
Model Studio



In Model Studio, the Neural Network node starts by standardizing all the input variables. This is also known as scaling the inputs. The default standardization method, which is midrange, scales the variables so that they have a midrange of zero, a minimum of -1, and a maximum of 1. The midrange is the mean of the smallest value and the largest value.

Other available methods are Z-score standardization and no standardization.

2. Find the weight estimates.

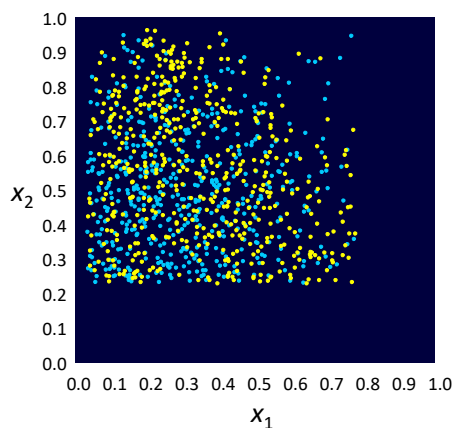
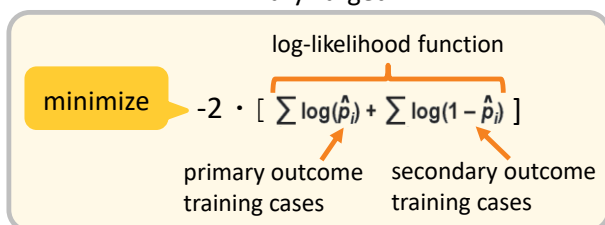
$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

Binary Target



The primary task is to find the weight estimates that result in accurate predictions. Compared to a regression model, a neural network has a higher number of values to be estimated and the relationship among the weights is more complex. For a binary target, the weight estimation process is driven by an attempt to minimize the negative of two times the log-likelihood function. The log-likelihood function is the sum of the log of the primary outcome training cases plus the sum of the log of the secondary outcome training cases.

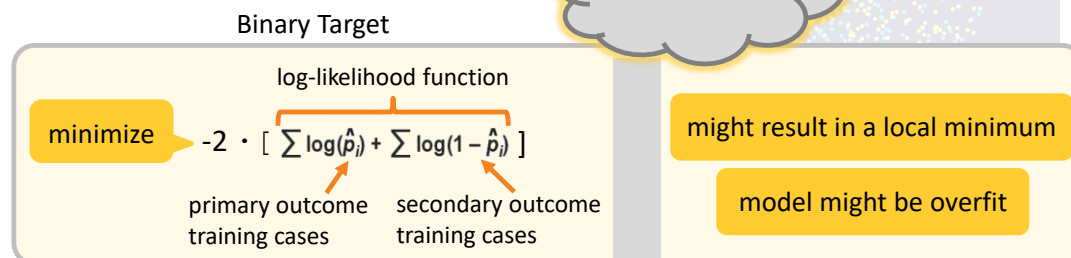
2. Find the weight estimates.

$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$



Unfortunately, with a neural network model, this minimization process has two complications.

First, the minimization process might result in a local minimum, which is a point that is smaller than the surrounding points, instead of the global minimum (which is the smallest point over all).

Second, neural networks tend to create overfit models. A technique called early stopping usually overcomes these difficulties and produces a reasonable model. Early stopping can also be called stopped training. You learn about early stopping later in this lesson.

2. Find the weight estimates.

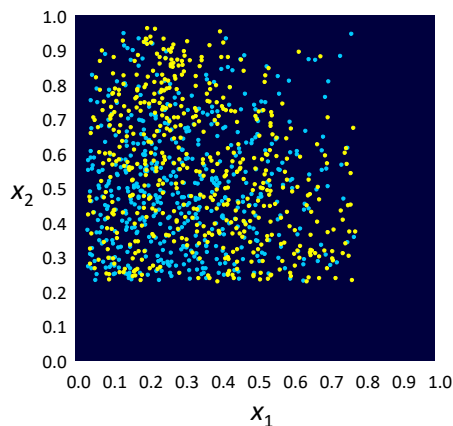
$$\text{logit}(\hat{p}) = -0.5 + -2.6 H_1 + -1.9 H_2 + -0.63 H_3$$

$$H_1 = \tanh(-1.8 + 0.25 x_1 + -1.8 x_2)$$

$$H_2 = \tanh(2.7 + 2.7 x_1 + -5.3 x_2)$$

$$H_3 = \tanh(-5.0 + 8.1 x_1 + 4.3 x_2)$$

target-inputs associations can be complex



Is the relationship between X_2 and the target positive or negative?

The weight estimates have now been inserted into the prediction formula. Keep in mind that, even in a simple neural network model, the associations between the inputs and the target can be complex, and the model can be difficult to interpret.

In this example, can we determine whether the relationship between X_2 and the target is positive or negative? Clearly, we cannot. Notice that two of the weights estimated for X_2 are negative but one is positive.

3. Obtain a prediction.

$$\text{logit}(\hat{p}) = -0.5 + -2.6 H_1 + -1.9 H_2 + -0.63 H_3$$

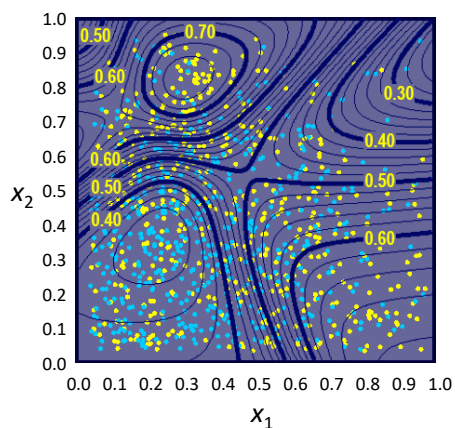
$$H_1 = \tanh(-1.8 + 0.25 x_1 + -1.8 x_2)$$

$$H_2 = \tanh(2.7 + 2.7 x_1 + -5.3 x_2)$$

$$H_3 = \tanh(-5.0 + 8.1 x_1 + 4.3 x_2)$$

Logistic Function

$$\hat{p} = \frac{1}{1 + e^{-\text{logit}(\hat{p})}}$$



After the prediction formula is established, obtaining a prediction is strictly a matter of inserting the input measurements into the hidden units. For example, to obtain a probability estimate, given values of x_1 and x_2 , the estimated prediction formula provides a logit, which is then inserted into the logistic function.

Standardization Methods

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Learning Process



Copyright © SAS Institute Inc. All rights reserved.

Learning Process

numerical
optimization

$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

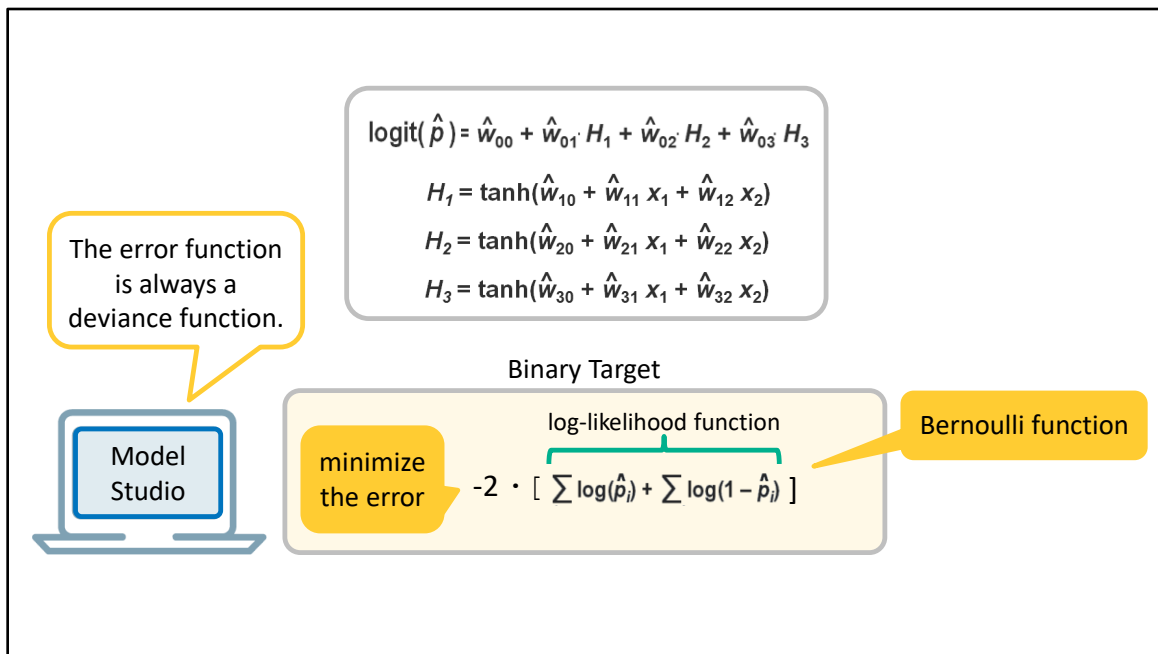
$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$



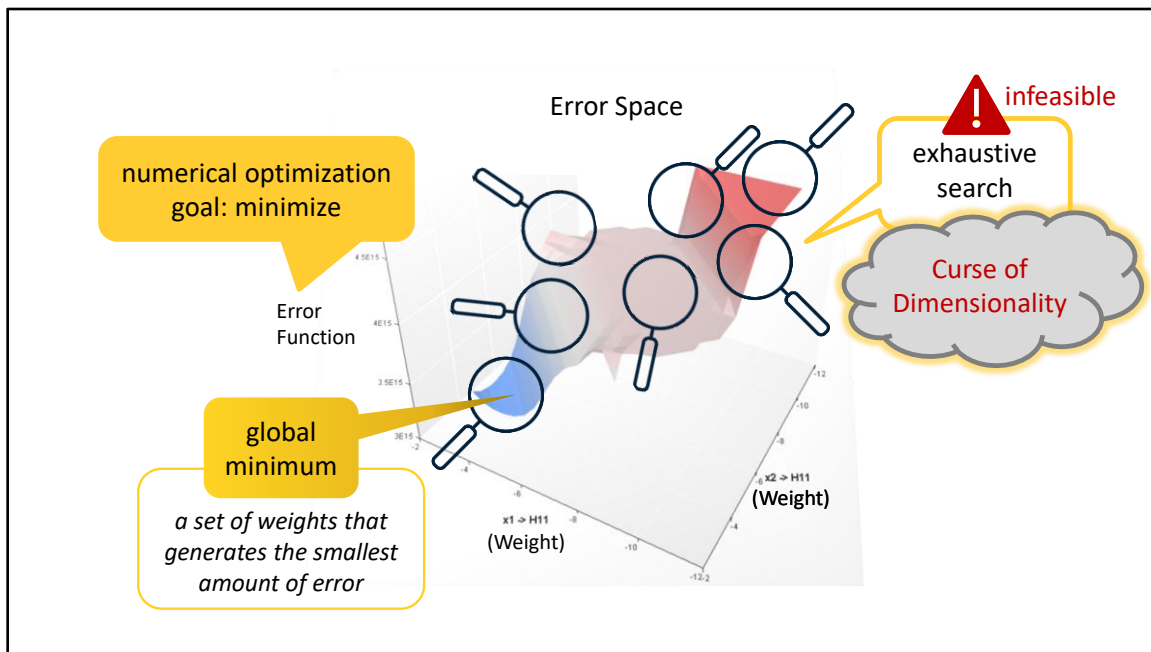
learning

using an optimization
method to estimate the
weights that minimize
the error function

A neural network uses a numerical optimization method that you specify to estimate the weights that minimize the error function. This process is called learning.



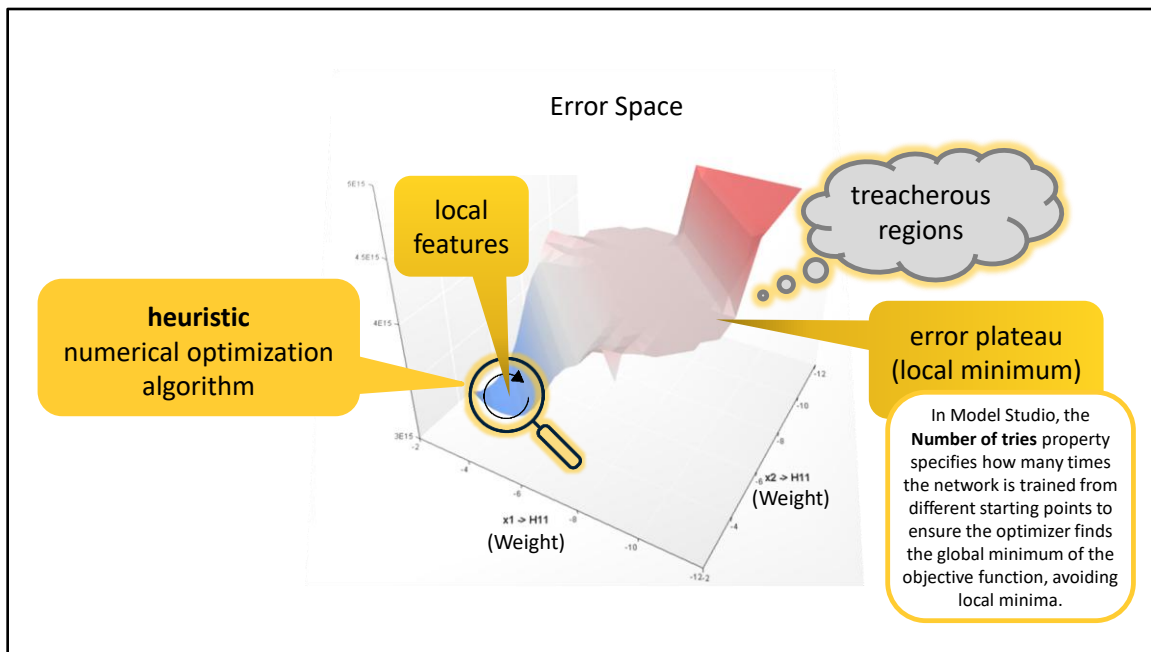
In this example, shown earlier, weight estimates for a binary target are produced by minimizing the error, which is -2 times the log-likelihood. This particular error function is known as the Bernoulli function. In Model Studio, the error function is always a deviance function.



A three-dimensional plot of an error space is shown here. The error function and the weights in the model define the surface of the error space. The error space can also be called an error surface.

Remember that the goal of numerical optimization is to minimize the error function. This is the same as finding the lowest point in the error space, which is called a global minimum. Formally, a *global minimum* is a set of weights that generates the smallest amount of error.

A simple strategy to ensure that a global minimum has been found is an exhaustive search of the error space. Unfortunately, the curse of dimensionality quickly makes this method infeasible.



It is more efficient to use a heuristic numerical optimization algorithm to search for the optimal weight estimates. A heuristic is a "shortcut" that is known to work well in most situations. Numerical optimization algorithms use **local** features of the error surface to decide how to proceed.

However, using local features makes these algorithms vulnerable to getting stuck in treacherous regions of the error surface that result in little improvement. These regions can be referred to as *error plateaus* or local minima.

The learning process stops when an error plateau is reached, as any further movements yield very little improvement in the error.

Convergence Criteria

- The specified error function stops improving.
- The gradient has no slope.
- The magnitude of the weights stops changing substantially.



The learning process stops when the numerical optimization algorithm determines that it has learned the data well enough (that is, within a certain margin of error).

The decision to stop is based on meeting at least one of three convergence criteria:

The specified error function stops improving.

The gradient has no slope (in other words, the rate of change of the error function with respect to the weights is zero).

Or, the magnitude of the weights stops changing substantially.

Iterative Updating in Numerical Optimization

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Numerical Optimization Methods in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Deviance Measures in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Optimization Methods



Copyright © SAS Institute Inc. All rights reserved.

Question: Optimization Methods

Optimization methods are used to efficiently search the complex landscape of the error surface to find an error maximum.

- a. True
- b. False

Answer: Optimization Methods

Optimization methods are used to efficiently search the complex landscape of the error surface to find an error maximum.

a. True

☒ b. False

CA: b

FB: Optimization methods are used to efficiently search the complex landscape of the error surface to find an error **minimum**.

Essential Discovery Tasks for Neural Networks



Copyright © SAS Institute Inc. All rights reserved.

Essential Discovery Tasks for Neural Networks



Essential Discovery Tasks

- ✓ Select an algorithm.
- Improve the model.
- Optimize the complexity of the model.
- Regularize and tune the hyperparameters of the model.

The essential discovery tasks shown here for neural networks are the same as for decision trees, with one exception: building ensemble models is not listed. Ensembles are not considered for neural network models as often as they are for tree-based models.

As usual, after selecting the algorithm, we build a neural network using the default settings.

Demo: Building a Neural Network Using the Default Settings

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



4.2 Modifying the Model: Network Architecture



Improving the Neural Network Model



Copyright © SAS Institute Inc. All rights reserved.

Improving the Neural Network Model



Essential Discovery Tasks

- ✓ Select an algorithm.
- Improve the model. architecture
- Optimize the complexity of the model.
- Regularize and tune the hyperparameters of the model.

After building the neural network using the default settings, we move on to the next discovery task: improving the model. At this point, it is common to examine the neural network architecture. By modifying the architecture, we might be able to increase model flexibility and improve model performance.

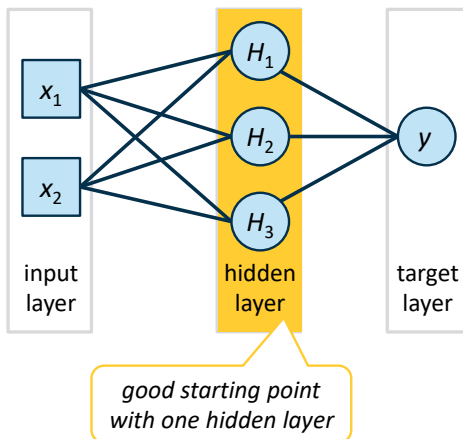
Neural Network Architectures



Copyright © SAS Institute Inc. All rights reserved.

Neural Network Architectures

Multilayer Perceptron



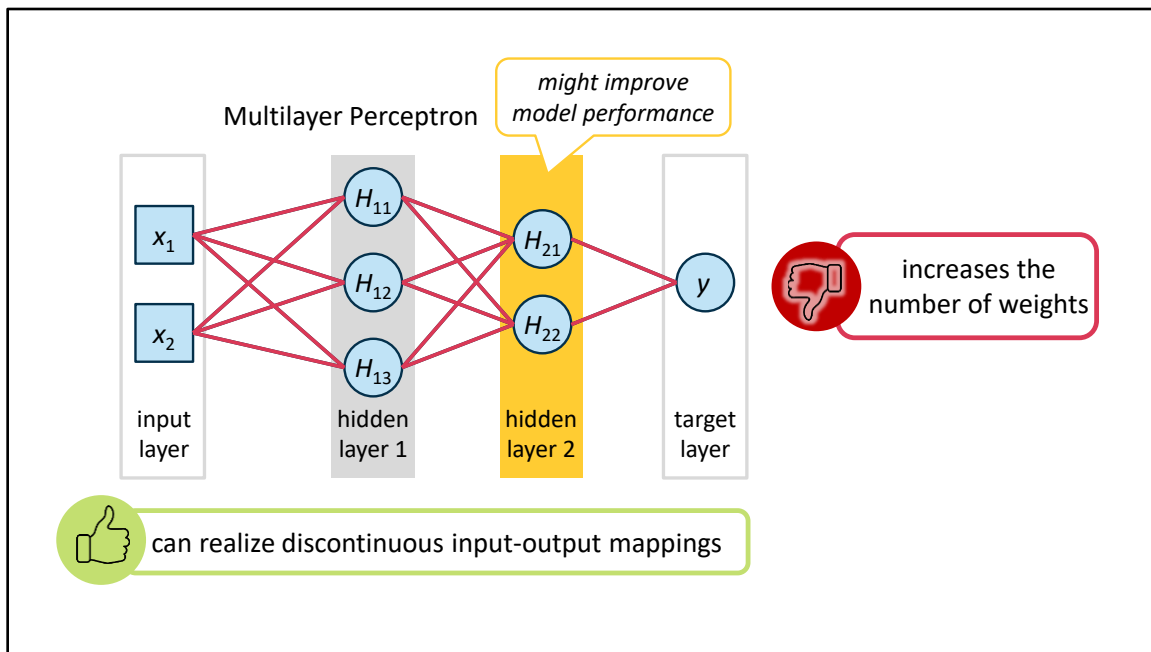
*might require a more flexible,
more complex architecture*

+ layers

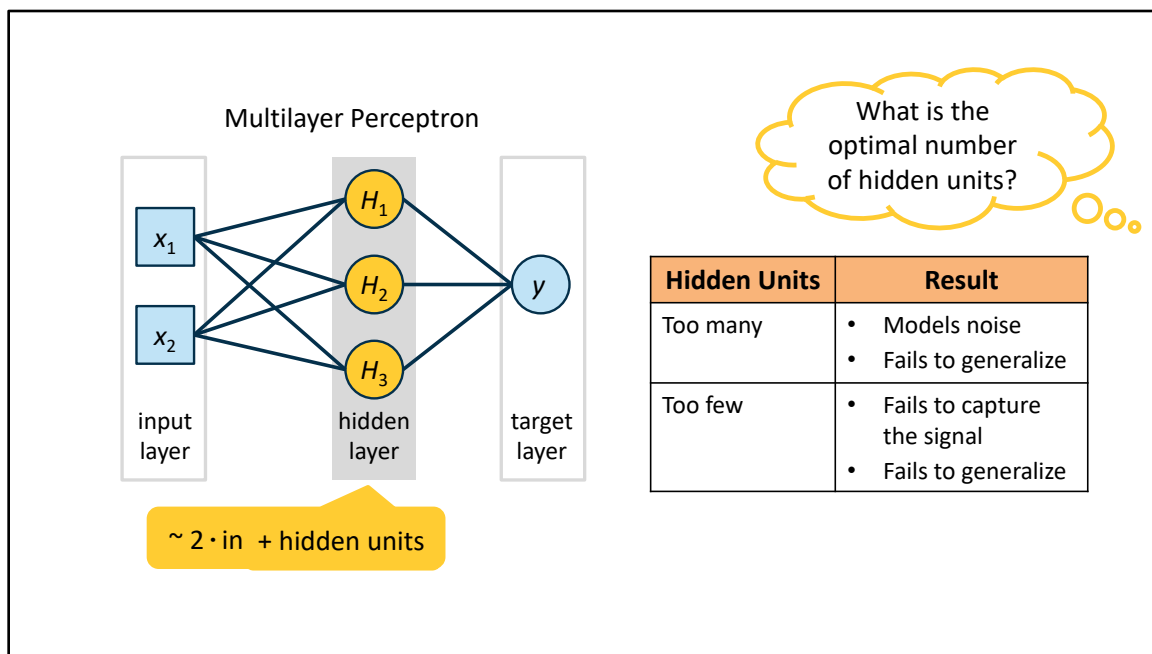
+ neurons in each layer

different connection types

You have already seen one neural network architecture: a basic MLP with one hidden layer. This architecture is a good starting point. However, some modeling situations require a more flexible, more complex neural network architecture. Neural networks can have additional layers, additional neurons in each layer, and different types of connections. In many situations, a basic MLP with one hidden layer is sufficient.

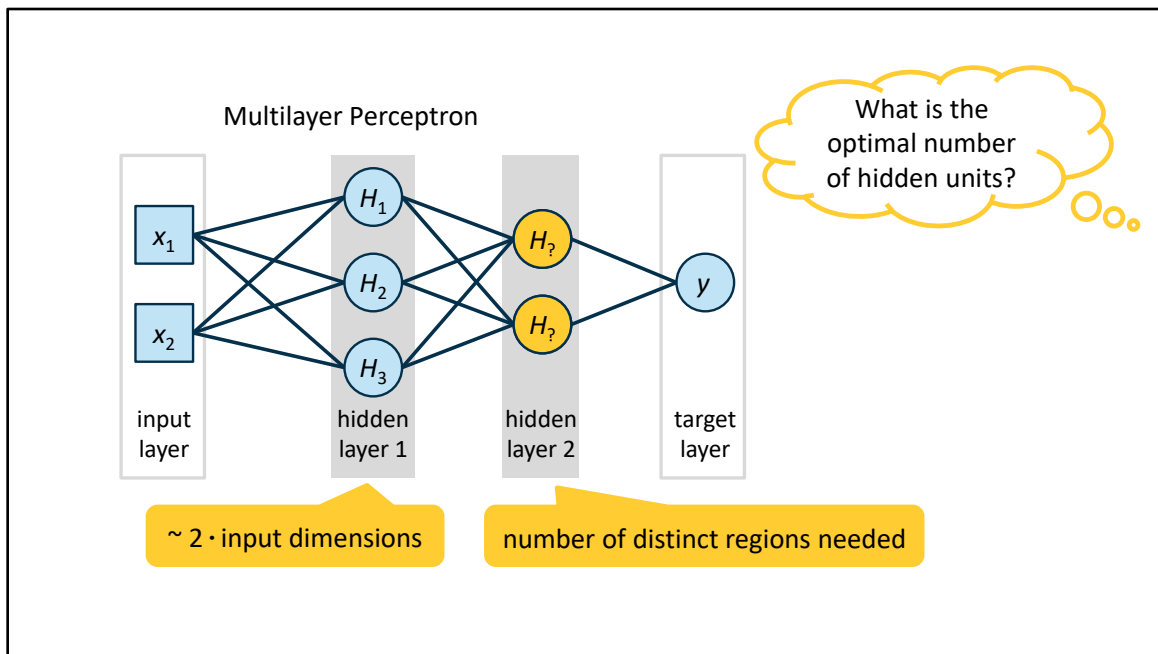


However, a second layer of hidden units can improve model performance. Specifically, adding a second hidden layer enables the MLP to realize discontinuous input-output mappings. On the other hand, adding a second hidden layer also significantly increases the number of weights.

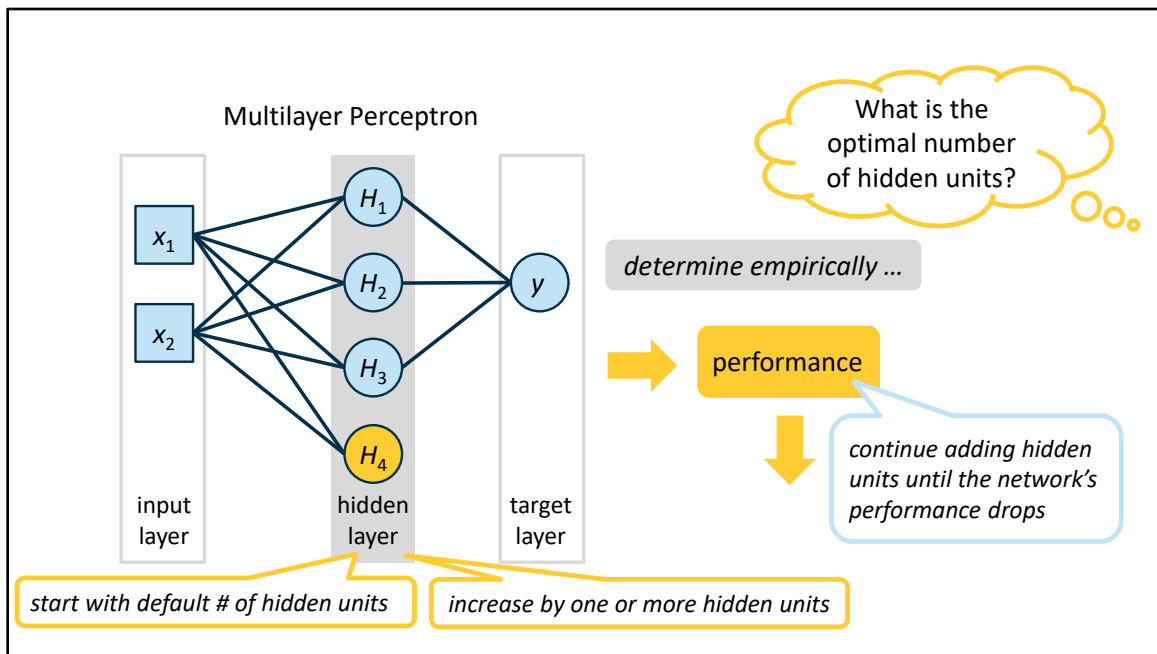


Adding hidden units can improve model performance. However, determining the optimal number of hidden units is more difficult than determining the optimal number of hidden layers. If the network has too many hidden units, it will model random variation, or noise, as well as the desired pattern, or signal. This means that the model will fail to generalize. Conversely, having too few hidden units will fail to adequately capture the underlying signal. Again, the model will fail to generalize.

The optimal number of hidden units is problem specific. Fortunately, there are guidelines to help you determine the number of hidden units. Research suggests that the number of units in the first hidden layer should be about twice the number of input dimensions.



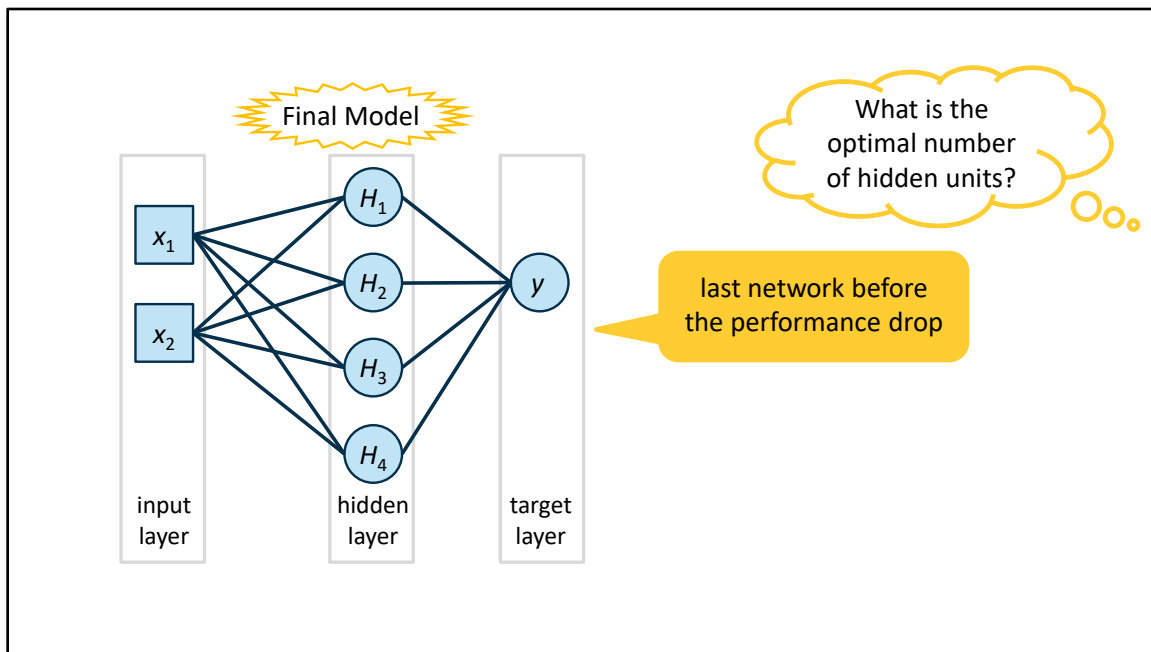
If a second hidden layer is required, then the number of hidden units in the second layer should reflect the number of distinct regions needed.



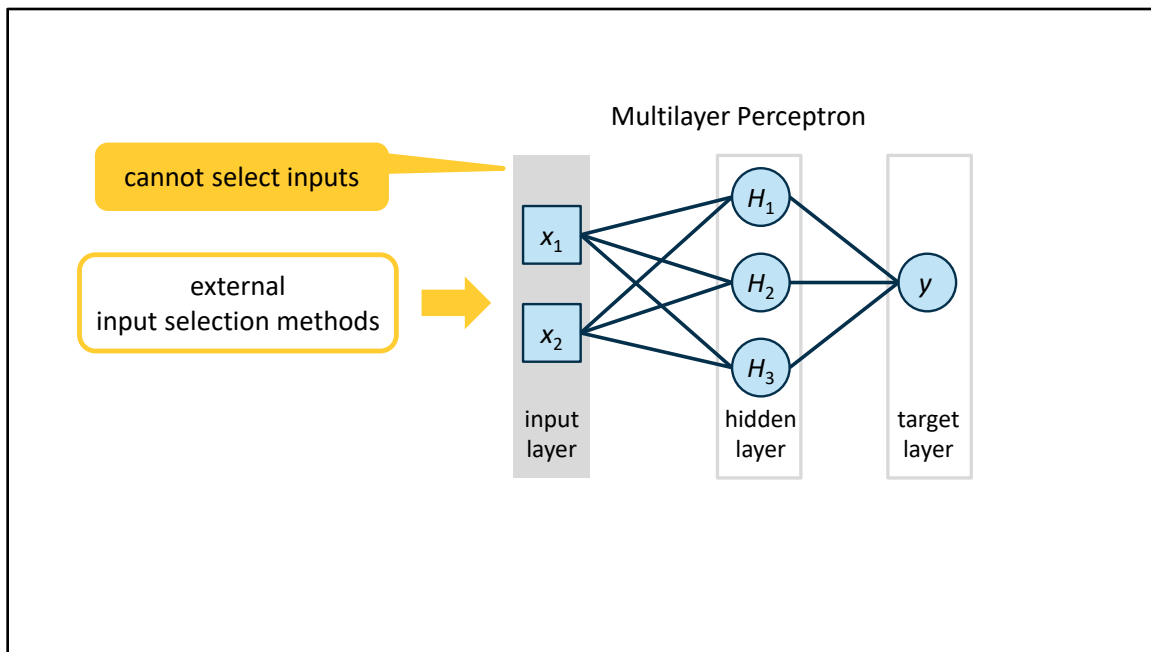
It might be best to determine the appropriate number of hidden units empirically, in the following way:

Start with a default neural network and measure its performance on some appropriate metric.

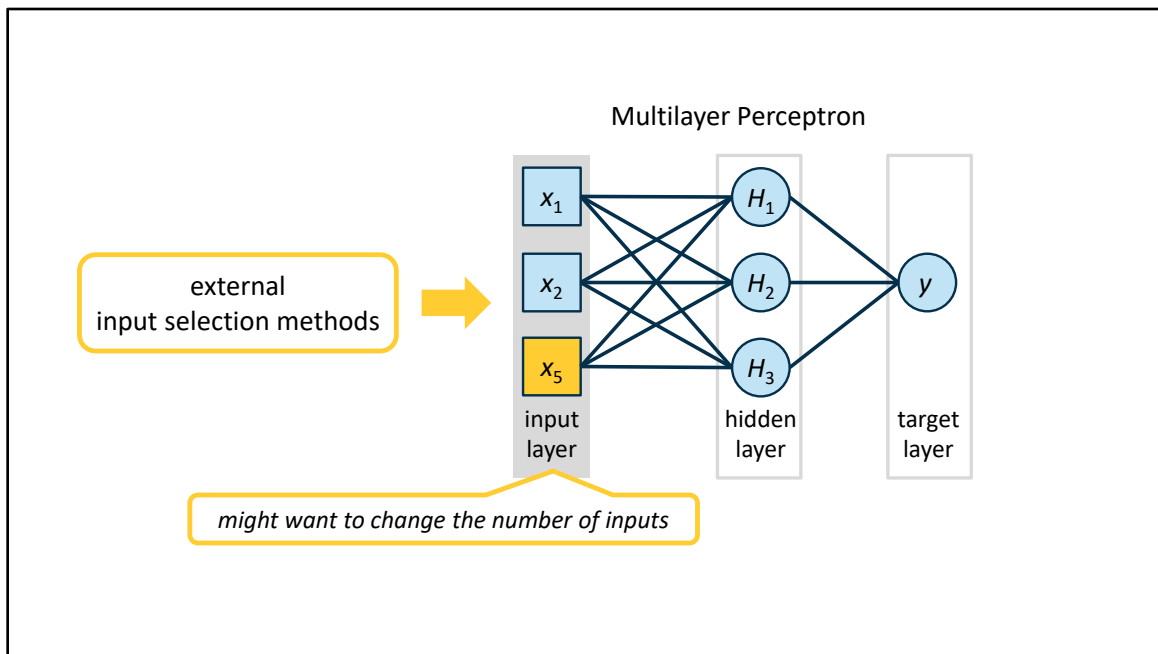
Then increase the number of hidden units by one and observe the impact on the network's fit. Continue adding hidden units until the network's performance drops.



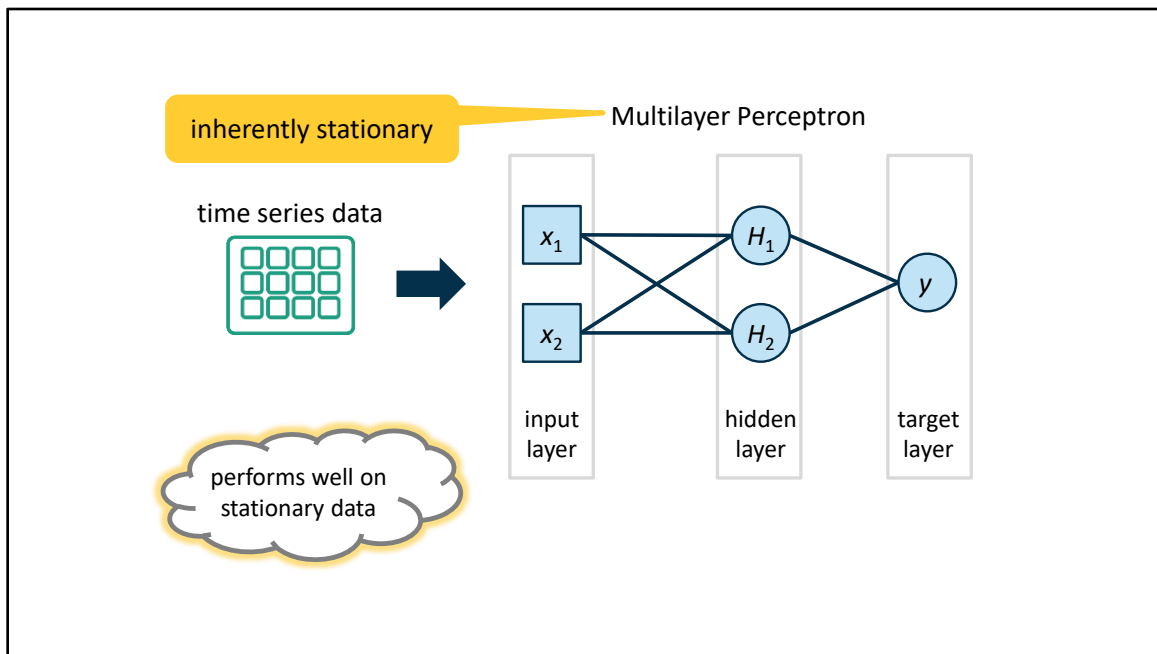
As the final model, select the last network generated before the drop in performance. This final model has the optimal number of hidden units.



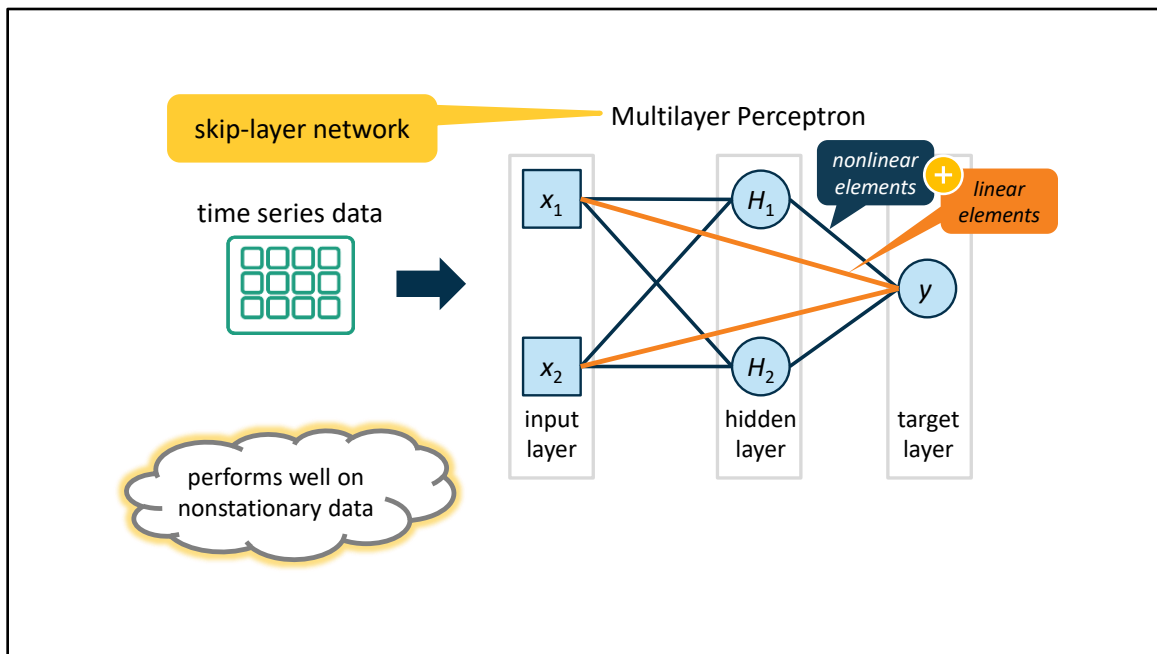
Unlike some other types of models, such as tree-based models, a neural network cannot select inputs. It is a good idea to try several external input selection methods to find the best combination of inputs for your model.



As a result, you might want to change the number of inputs in the input layer.



An MLP is inherently stationary. This means that it will tend to perform poorly when applied to nonstationary data (that is, data that has elements that change over time). An example of nonstationary data (also known as time series data), is monthly electricity usage per household.



When working with time series data, it can be helpful to add direct connections from the input layer to the target layer, bypassing the hidden layer or layers. The result is a special type of MLP known as a skip-layer network. This combines linear elements (that is, the direct connections) and nonlinear elements (that is, connections from the hidden layer) into a single neural network. An MLP is already a universal approximator, so there is often little to be gained by adding direct connections when you are not working with time series data.

In the diagram shown here, the skip-layer perceptron has two inputs and one hidden layer with two hidden units. However, a skip-layer perceptron can have a different number of inputs, hidden layers, and hidden units. The direct connections between the inputs and the target define a skip-layer perceptron.

Stationary versus Nonstationary Data

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Calculating the Number of Parameters

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Deep Learning

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Deep Learning Models



Copyright © SAS Institute Inc. All rights reserved.

Question: Deep Learning Models

Which of the following techniques do deep learning models use to overcome the computational challenges associated with multiple hidden layers?

- a. direct connections between the input layer and the target layer
- b. fast moving gradient-based optimizations
- c. a first hidden layer in which the number of hidden units is twice the number of inputs in the input layer
- d. standardized inputs

Answer: Deep Learning Models

Which of the following techniques do deep learning models use to overcome the computational challenges associated with multiple hidden layers?

- a. direct connections between the input layer and the target layer
- ☒ b. fast moving gradient-based optimizations
- c. a first hidden layer in which the number of hidden units is twice the number of inputs in the input layer
- d. standardized inputs

CA: b

FB: Deep learning models use fast moving gradient-based optimizations, such as Stochastic Gradient Descent, for this purpose.

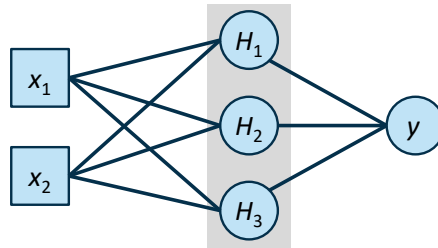
Activation Functions



Copyright © SAS Institute Inc. All rights reserved.

Activation Functions

ability to model nonlinear relationships



hidden unit
activation function

Remember that the strength of neural networks is their ability to model nonlinear relationships. The hidden unit activation function is the key contributor to the model's ability to model nonlinearities.

Many hidden unit activation functions are members of the sigmoid family.

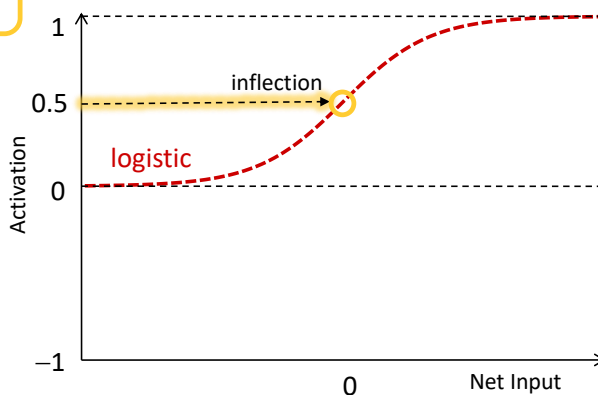
$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

hidden unit
activation function



In each hidden unit, after the weighted inputs and the bias are combined, the hidden unit's net input is passed through an activation function. Many of the available hidden unit activation functions are members of the sigmoid family.

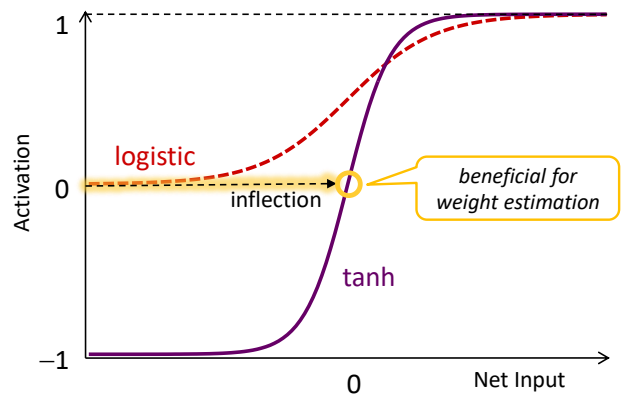
A well-known member of the sigmoid family is the logistic function. The logistic activation function constrains its output to the range of 0 to 1 with the inflection point at 0.5.

$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

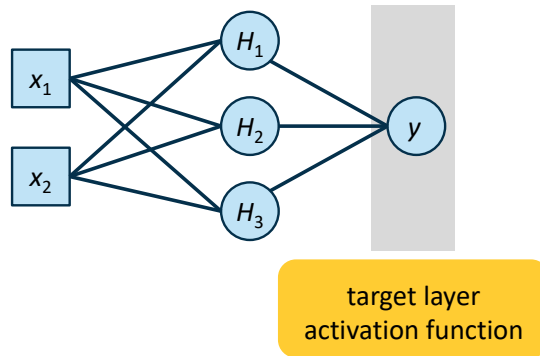
$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$



Although the logistic activation function was used as the hidden unit activation function in early neural network research, many other sigmoidal activation functions exist. In Model Studio, the Neural Network node uses the hyperbolic tangent (or tanh) as the default activation function. The hyperbolic tangent ranges from -1 to 1. This means that the inflection point is at 0 rather than at 0.5, as it is in the logistic sigmoid. This offers a small advantage during weight estimation.



Activation functions can also be used in the target layer to model outcome distributions for different types of targets.

binary target

logistic function

$$\log\left(\frac{\hat{p}}{1-\hat{p}}\right) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

In this example, the target is binary. The target layer link function (also known as the target layer activation function) is the logistic function, which is typically used with a binary target. The logistic function is the inverse of the logistic activation function. The logistic activation function constrains its output to the range of 0 to 1, so it is an ideal candidate for generating probability estimates when used in the target layer.

Hidden Layer Activation Functions in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Target Layer Activation Functions and Error Functions in Model Studio

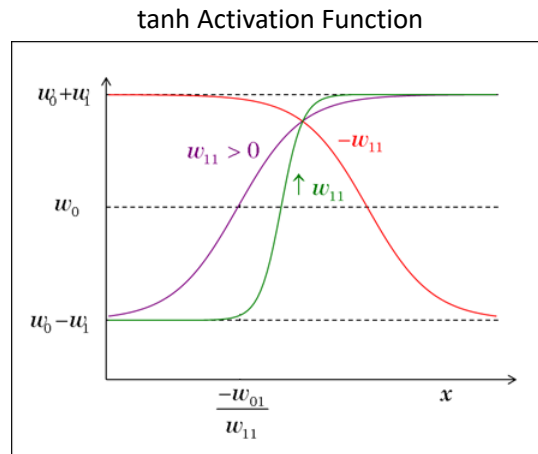
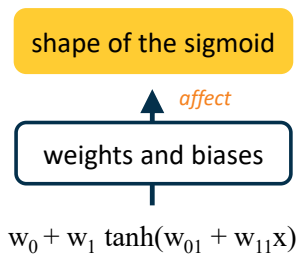
To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Shaping the Sigmoid

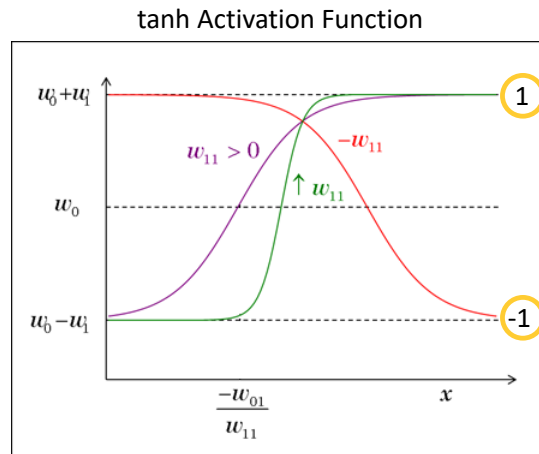
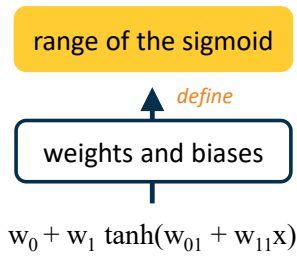


Copyright © SAS Institute Inc. All rights reserved.

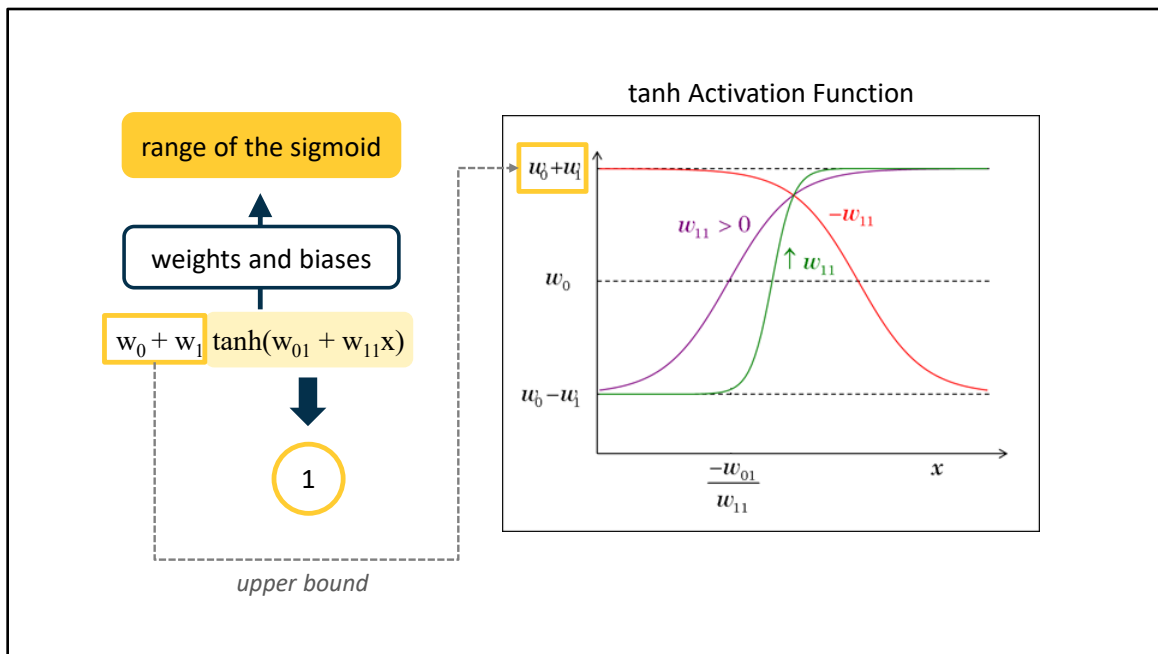


In a neural network, the weights and biases affect the shape of the activation function's sigmoid.

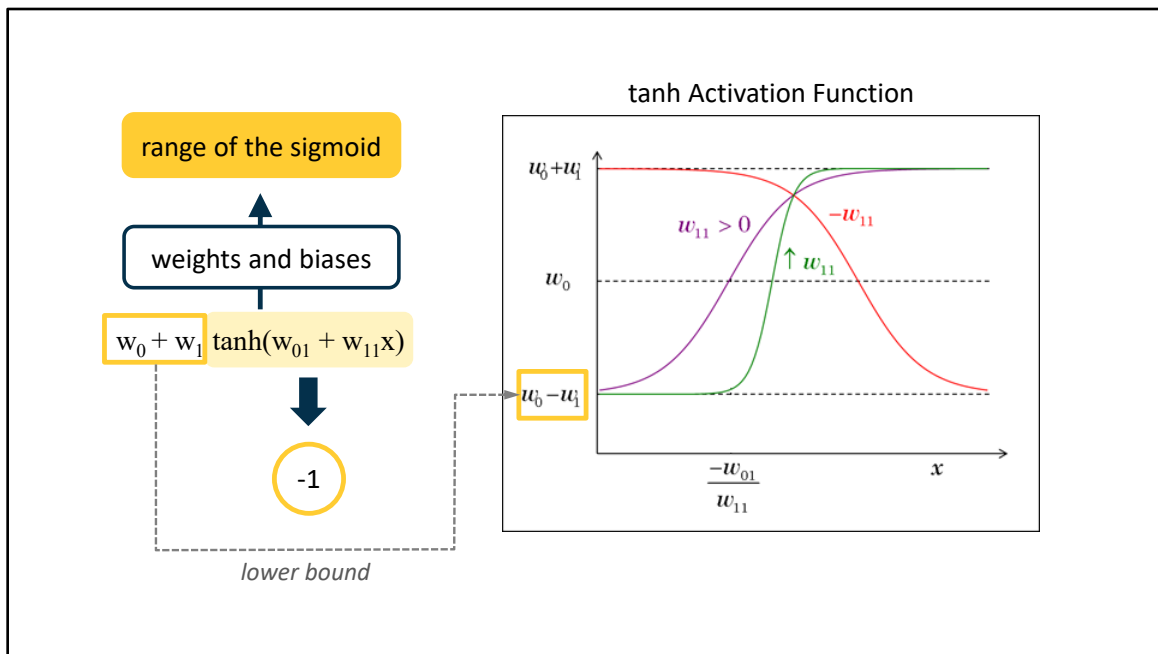
For example, this plot shows sigmoids for the tanh activation function in a very basic MLP. This MLP has a single input (x) and one hidden unit (represented as tanh of the quantity $w_{01} + w_{11}x$).



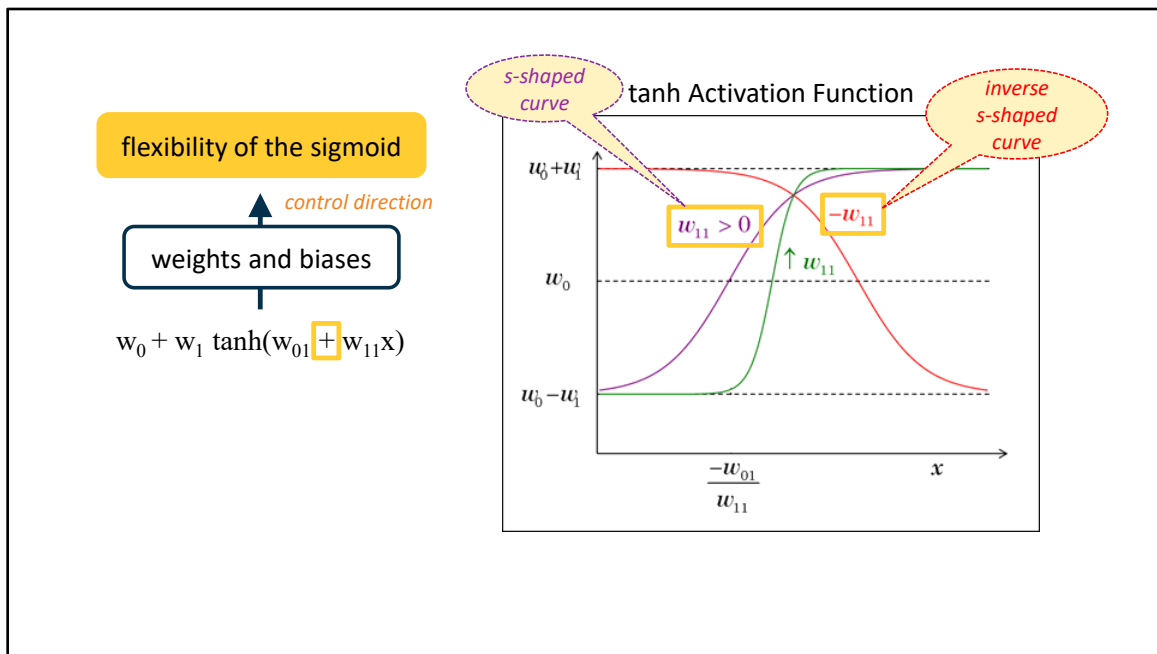
The weights and biases define the *range* of the sigmoid. Remember that the range of the tanh function is 1 to -1.



When tanh returns the value 1, the upper bound of its activation range is given by the output unit's bias plus its weight. In this example, the upper bound equals $w_0 + w_1$.



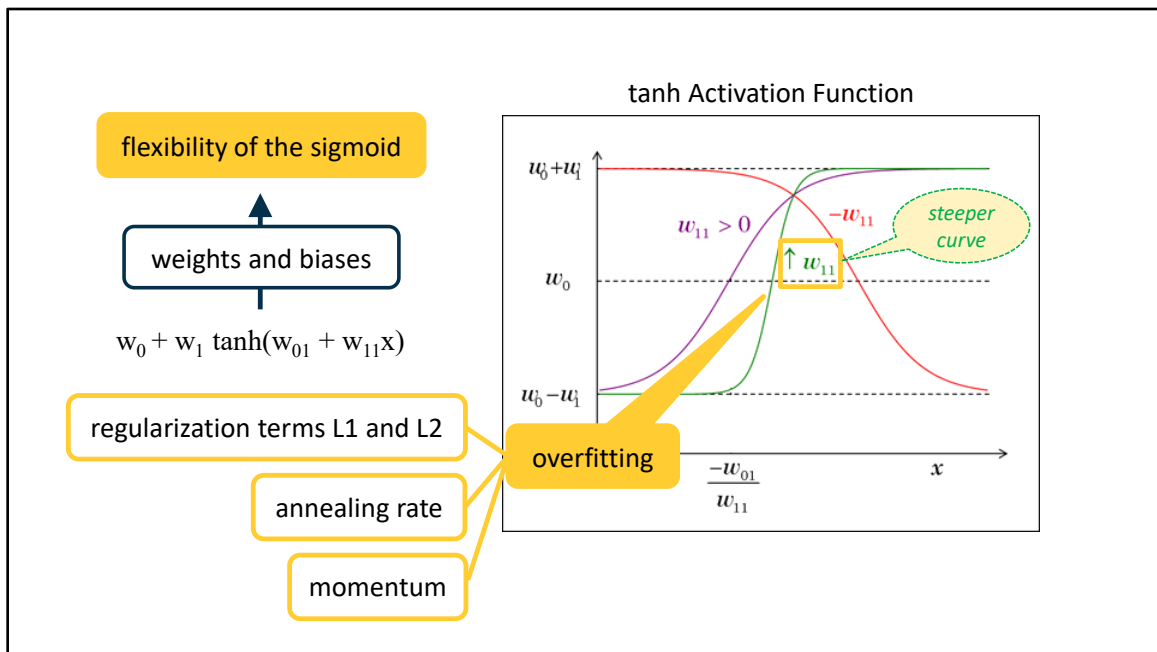
Conversely, the minimum activation value of the hyperbolic tangent function is -1, which means that the lower bound of the hyperbolic tangent's activation range is the output unit's bias minus its weight. In this example, the lower bound is $w_0 - w_1$.



The weights and biases also give the sigmoid surfaces their *flexibility*. The sign of the weight associated with the input x controls the direction of the sigmoid.

Positive weight values produce the familiar s-shaped curve (shown here by the purple line labeled $w_1 > 0$).

Negative weight values flip the sigmoid horizontally (shown by the red line labeled $-w_{11}$).



The larger the absolute value of the weight, the steeper the curve (shown by the green line labeled by an up arrow followed by w_{11}). Steep sigmoids are often thought to be responsible for overfitting. Overfitting is controlled by the regularization terms L1 and L2, the annealing rate, and the momentum, which you learn about later.

Demo: Modifying the Neural Network Architecture

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



4.3 Modifying the Model: Network Learning and Optimization



Optimizing the Complexity of a Neural Network Model



Copyright © SAS Institute Inc. All rights reserved.

Optimizing the Complexity of a Neural Network Model



Essential Discovery Tasks

- ✓ Select an algorithm.
- ✓ Improve the model.
 - Optimize the complexity of the model.
 - Regularize and tune the hyperparameters of the model.

Now that we have changed the model architecture, in an attempt to improve the model, we're ready to optimize the complexity of the model.

$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

optimizing complexity

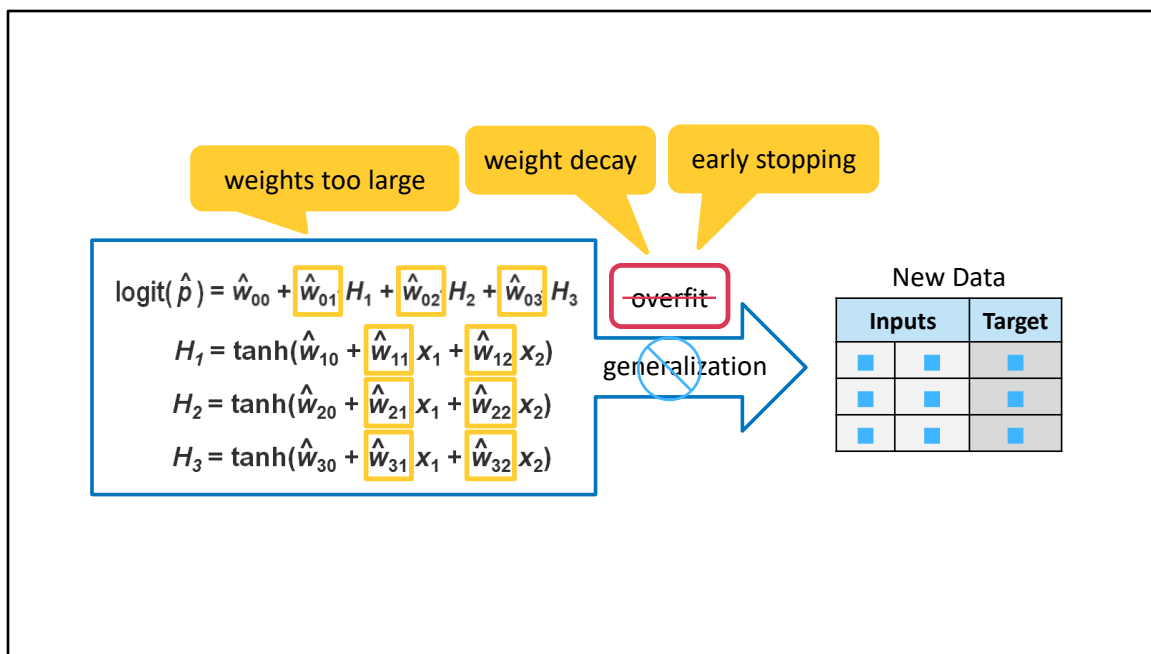


adding more terms
or rules



controlling the magnitude
of the weights

For neural networks, optimizing complexity does not involve adding more terms or more rules to the model. Instead, optimizing the complexity of a neural network involves controlling the magnitude of the weights.



If the weights grow too large, the model will be overfit and will not generalize well.

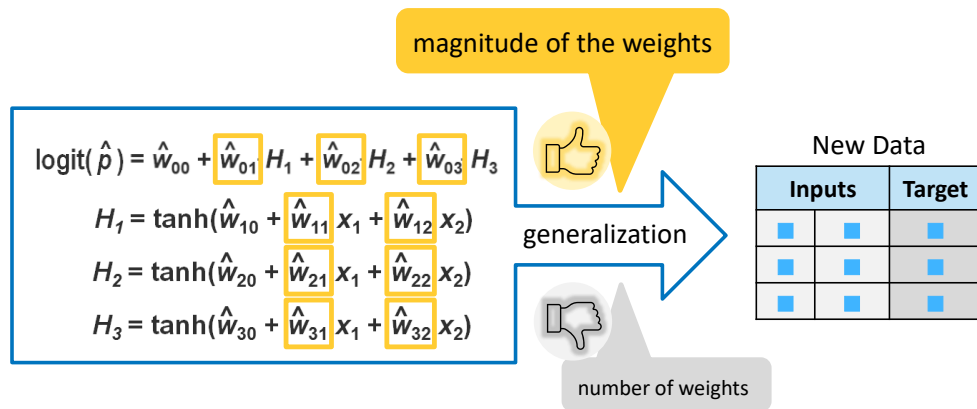
The two main methods of avoiding overfitting are weight decay and early stopping. These methods are often used together.

Weight Decay

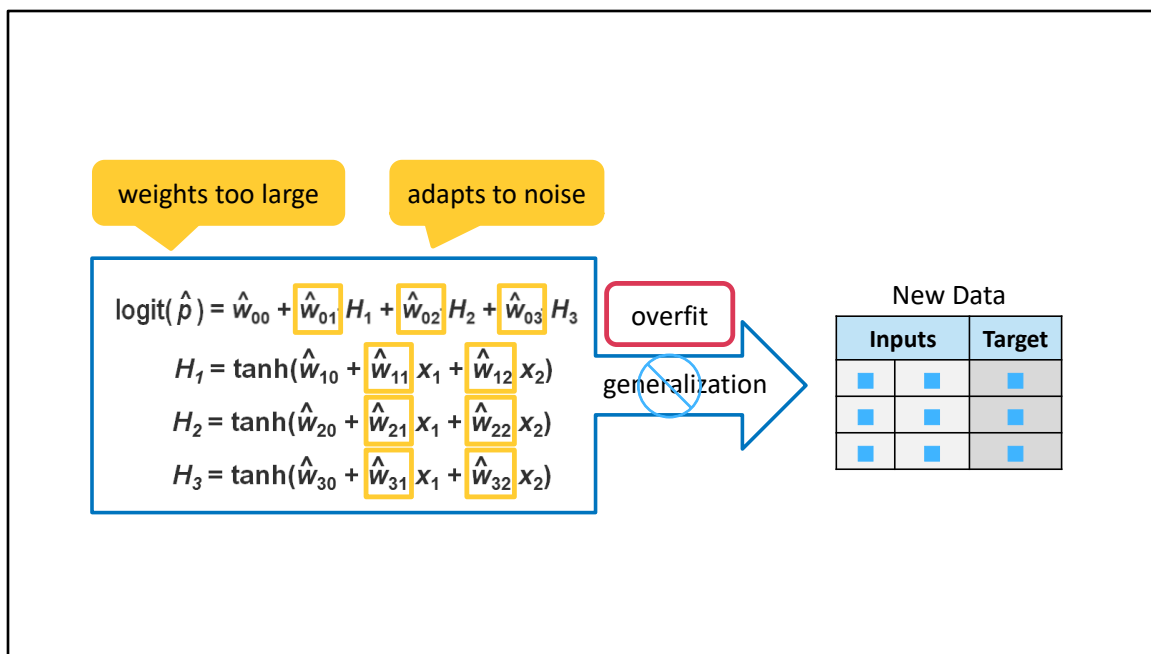


Copyright © SAS Institute Inc. All rights reserved.

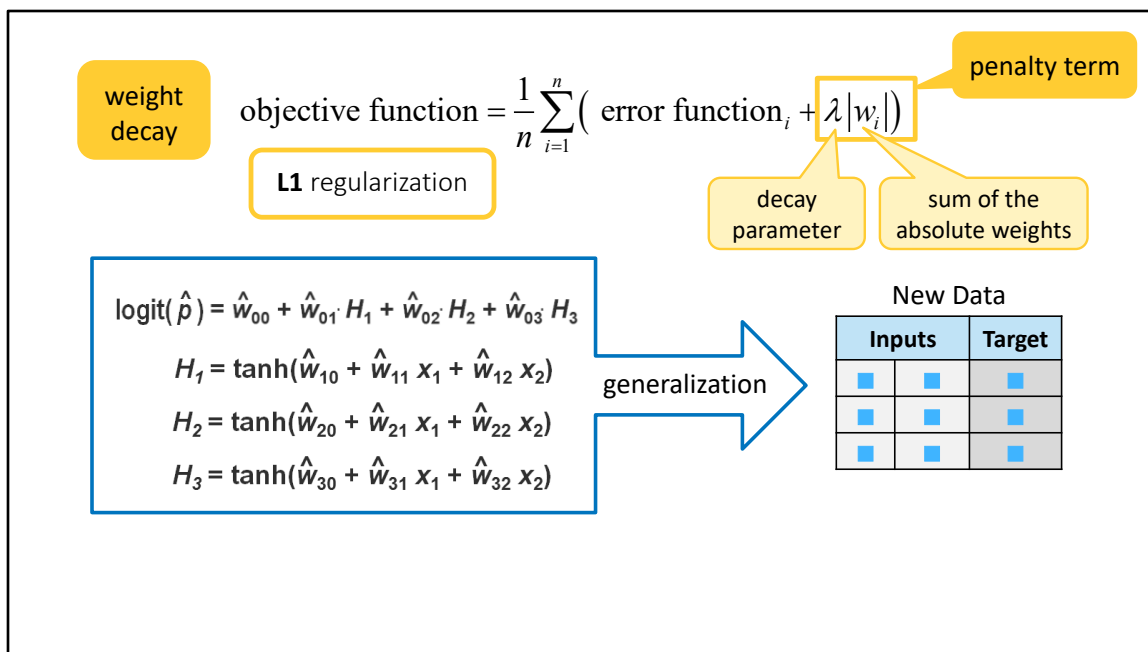
Weight Decay



Model generalization depends more on the magnitude of the weights than on the number of weights.

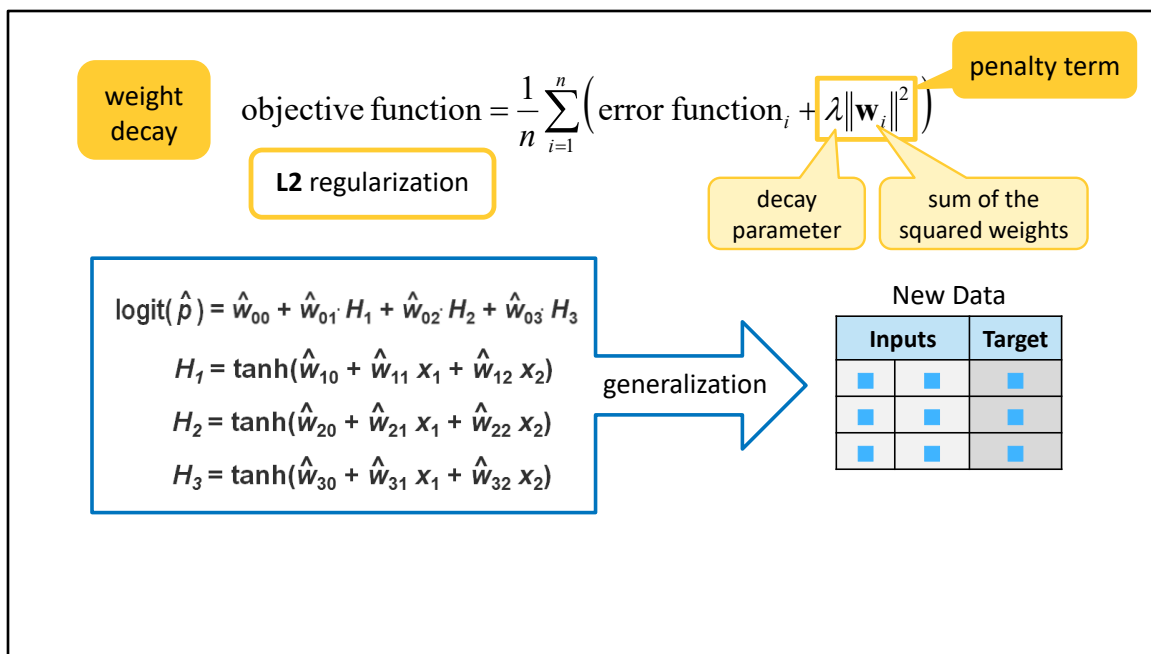


Very large magnitude weights tend to generate an irregular fit to the data because the model adapts to noise (that is, random variation). In other words, large weights are responsible for overfitting.



To avoid overfitting, we use a weight decay method with one or more regularizations. Regularizations are methods to help keep the sigmoids from becoming too steep. L1 regularization shrinks the weights toward zero.

To prevent the weights from growing too large, the weight decay method penalizes large weights by adding a term at the end of the objective function. This penalty term is the product of lambda (the decay parameter) and the sum of the absolute weights. The decay parameter controls the relative importance of the penalty term. Lambda commonly ranges from zero to one, although Model Studio does allow for values larger than 1.



L2 regularization is similar to L1 regularization in that both methods penalize the objective function for large network weights.

Instead of summing the absolute weights in the penalty term, L2 uses the sum of the squared weights. Here the penalty term is the product of lamda (the decay parameter) and the sum of the squared weights.

weight
decay

$$\text{objective function} = \frac{1}{n} \sum_{i=1}^n \left(\text{error function}_i + \lambda \|\mathbf{w}_i\|^2 \right)$$

penalty term
too large

The advantages outweigh the risks.

$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

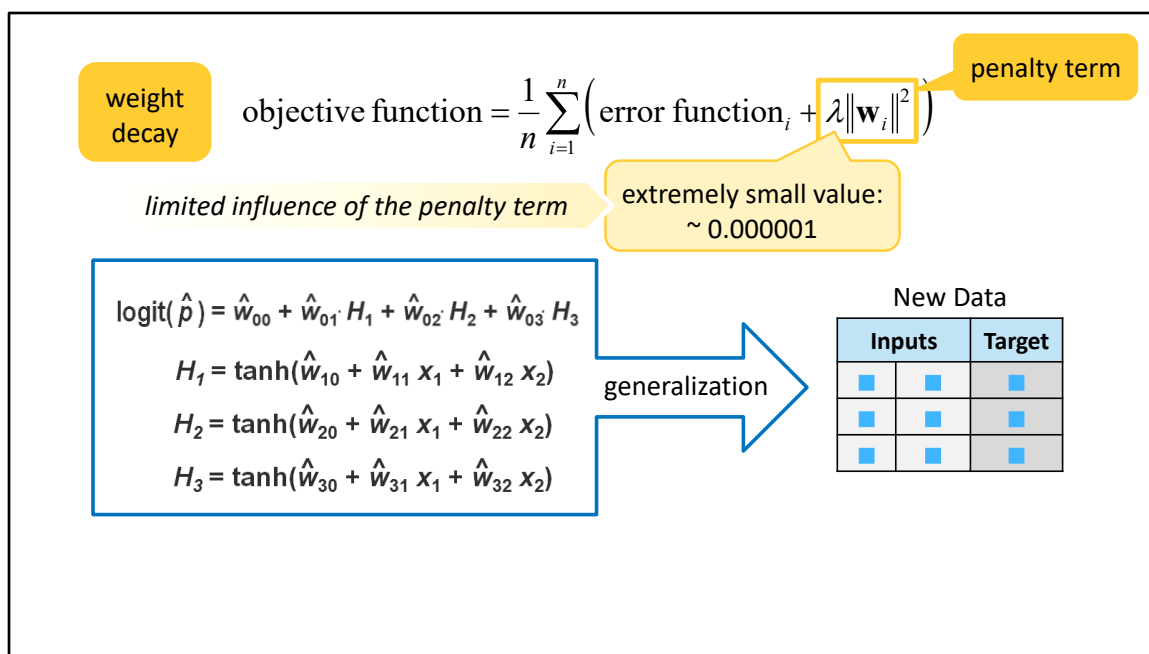
underfit

generalization

New Data

Inputs		Target
■	■	■
■	■	■
■	■	■

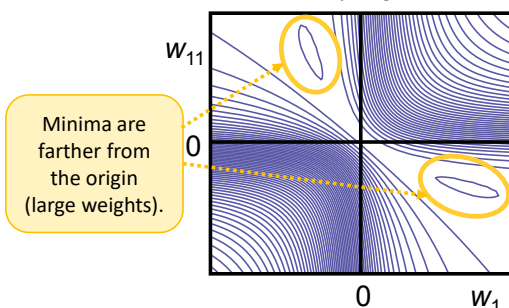
Specifying too large a penalty term risks the model under-fitting the data. However, the advantages of weight decay far outweigh its risks.



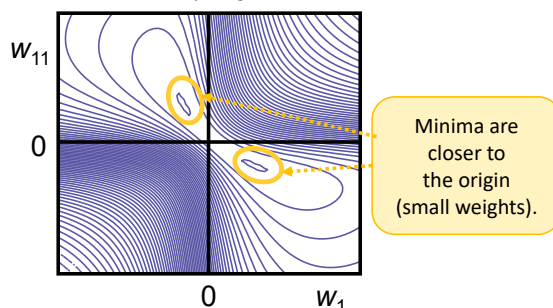
In addition, the influence of the penalty term is usually limited by selecting an extremely small value for the decay parameter. In many published studies, the magnitude of the decay parameter (lambda) is on the order of 0.000001.

$$\text{objective function} = \frac{1}{n} \sum_{i=1}^n \left(\text{error function}_i + \lambda \|\mathbf{w}_i\|^2 \right)$$

Objective Function
without Weight Decay
 $\lambda = 0$



Objective Function
with Weight Decay
 $\lambda > 0$



The graph on the left shows the objective function without weight decay. The graph on the right shows the objective function with weight decay.

In both graphs, the horizontal axis shows the magnitude of the weight w_1 , and the vertical axis shows the magnitude of the weight w_{11} .

In the graph on the left, with no weight decay, the minima are farther from the origin, so the weights used to find the minima are large.

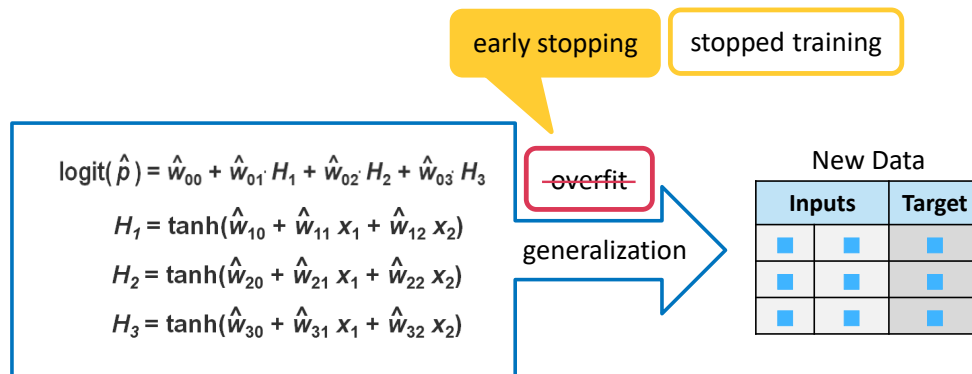
In the graph on the right, the minima are constrained closer to the origin, so the weights used to find the minima are smaller (thus, less likely to overfit).

Early Stopping



Copyright © SAS Institute Inc. All rights reserved.

Early Stopping



Early stopping, also called stopped training, is another method of avoiding overfitting by preventing the weights from growing too large.

Single model is estimated while building a neural network with fixed architecture.

$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

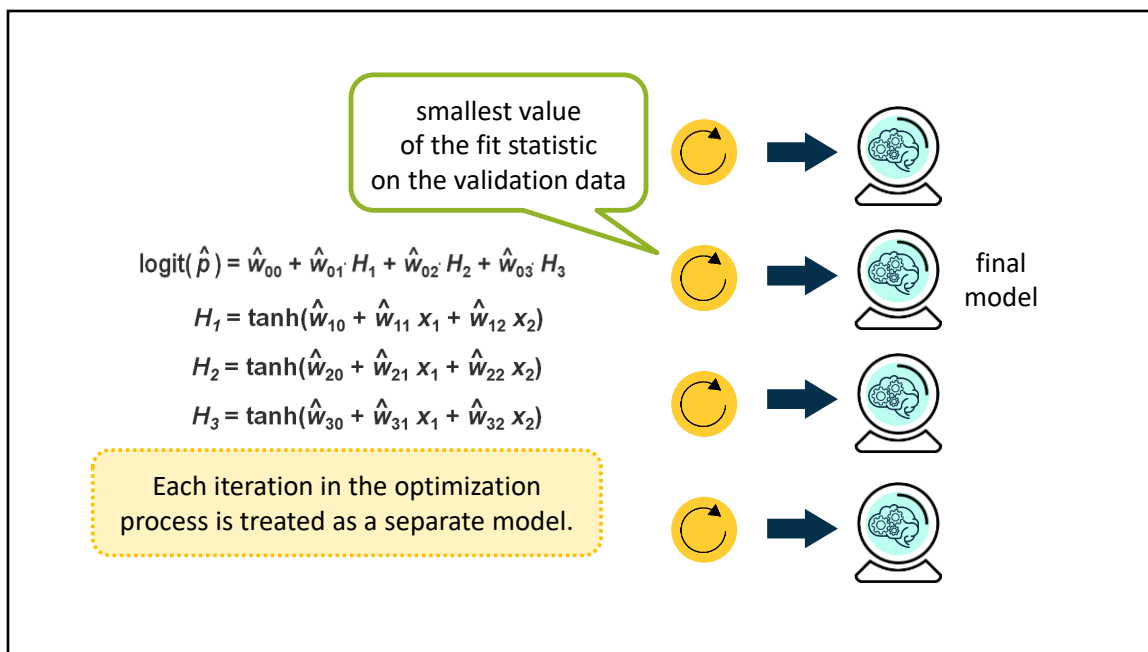
$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

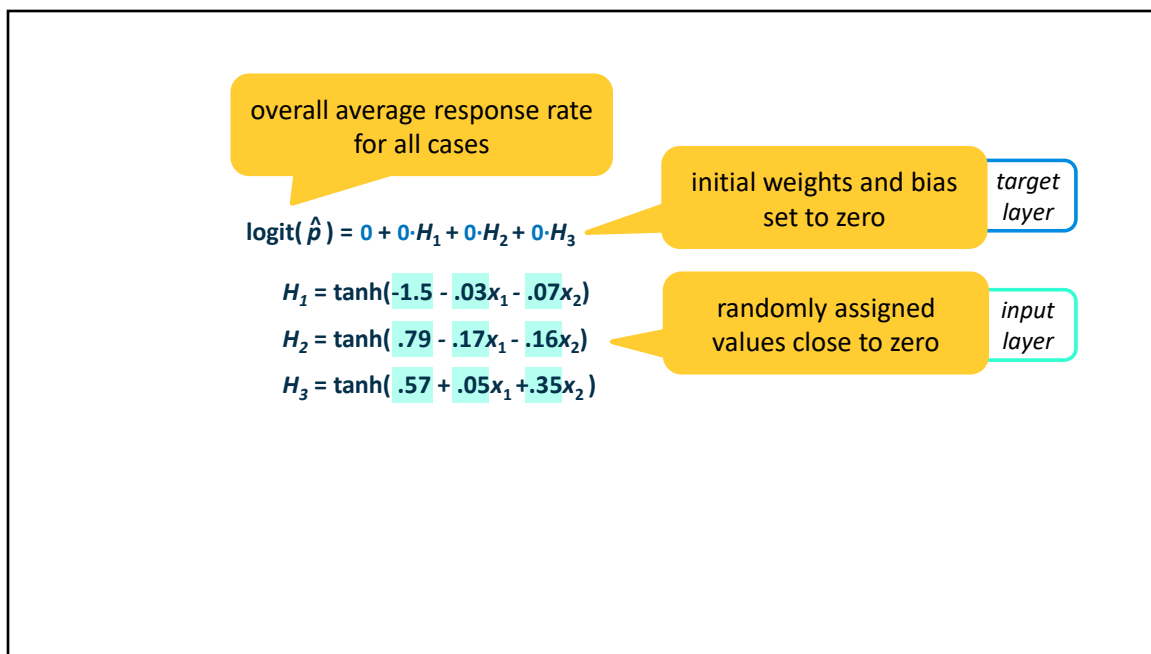


The weight magnitude changes complexity.

When you build a neural network, a single model is estimated, and the magnitude of the weight coefficients allows the complexity of that model to change.



SAS Viya treats each iteration in the optimization process as a separate model. The iteration with the smallest value of the selected fit statistic on the validation data is chosen as the final model.



At the beginning of the optimization process, the neural network model is set up to predict the overall average response rate for all cases. To do so, all weights and the bias in the target layer are set to zero, as shown in the equation. Next, the weights and the biases of the input layer are randomly assigned values close to zero (some positive and some negative).

updating the weight estimates such that ...

$$\text{objective function} = \frac{1}{n} \sum_{i=1}^n \left(\text{error function}_i + \lambda \|\mathbf{w}_i\|^2 \right)$$

$$\text{logit}(\hat{p}) = 0 + 0 \cdot H_1 + 0 \cdot H_2 + 0 \cdot H_3$$

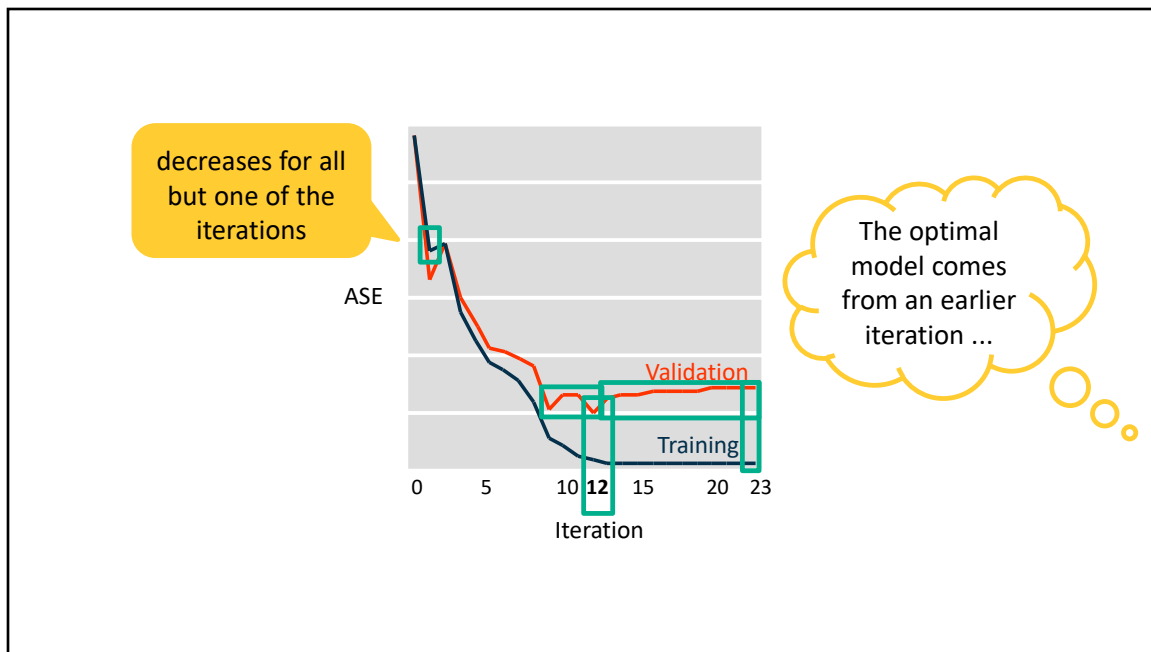
$$H_1 = \tanh(-1.5 - .03x_1 - .07x_2)$$

$$H_2 = \tanh(.79 - .17x_1 - .16x_2)$$

$$H_3 = \tanh(.57 + .05x_1 + .35x_2)$$

decreases the value

Training proceeds by updating the weight estimates in a manner that decreases the value of the objective function.



The graph plots the value of the objective function (here, the average squared error) on the Y axis and the iteration step on the X axis. In most of the early iterations, the objective function decreases for both the training and the validation data.

On the training data, notice that the ASE decreases for all but one of the iterations.

However, the same is not true of the ASE on the validation data set. Starting around iteration nine, changes to the weights sometimes improve ASE. However, for other iterations, changes to the weights cause ASE to increase, which indicates a decrease in model performance.

Finally at approximately iteration 12, the ASE continues to grow as the weight estimates are updated.

At the 23rd iteration, the weight estimates appear to be nearly stable and the model performance on the training data stops improving.

The philosophy of early stopping says that the optimal model comes from an earlier iteration where the objective function on the validation data has a smaller value than it does for the final iteration.

Iteration 12 provides the neural network where the complexity is optimized. Based on this neural network's performance on the validation data, it generalizes better than the model at iteration 23.

Question: Model Generalization



Copyright © SAS Institute Inc. All rights reserved.

Question: Model Generalization

For neural networks, model generalization depends more on the number of weights than on the magnitude of the weights.

- a. True
- b. False

Answer: Model Generalization

For neural networks, model generalization depends more on the number of weights than on the magnitude of the weights.

- a. True
- ☒ b. False

[CA: b]

FB: Model generalization depends more on the magnitude of the weights than on the number of weights. In fact, large weights are responsible for overfitting.

Question: Early Stopping



Copyright © SAS Institute Inc. All rights reserved.

Question: Early Stopping

When early stopping is used, where does the optimal model come from?

- a. one of the iterations that occur earlier than the final training iteration
- b. the iteration immediately before the final training iteration
- c. the final training iteration

Answer: Early Stopping

When early stopping is used, where does the optimal model come from?

- a. one of the iterations that occur earlier than the final training iteration
- b. the iteration immediately before the final training iteration
- c. the final training iteration

[CA: a]

FB: The philosophy of early stopping says that the optimal model comes from an earlier iteration where the objective function on the validation data has a smaller value than it does for the final iteration. The optimal model does not necessarily need to come from the iteration immediately before the final iteration.

Regularizing and Tuning the Hyperparameters of a Neural Network Model



Copyright © SAS Institute Inc. All rights reserved.

Regularizing and Tuning the Hyperparameters of a Neural Network Model



Essential Discovery Tasks

- ✓ Select an algorithm.
- ✓ Improve the model.
- ✓ Optimize the complexity of the model.
 - Regularize and tune the hyperparameters of the model.

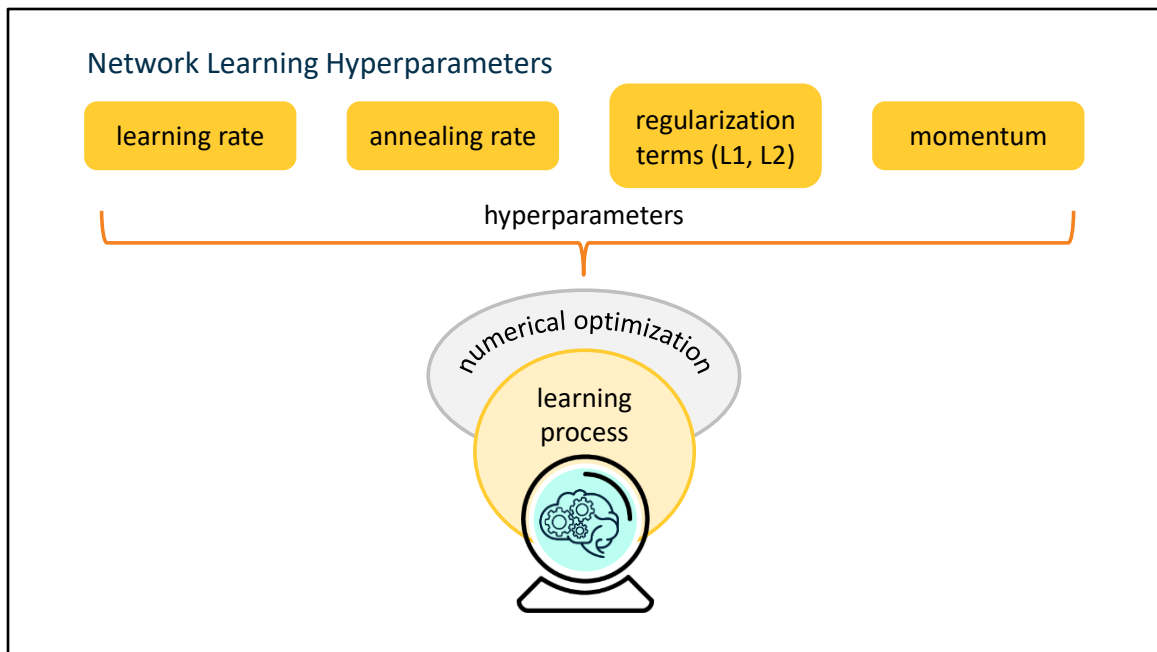
Autotuning is disabled in SAS Viya for Learners, and selecting it results in an error.

We are now at the last discovery task: Regularize and tune the hyperparameters of the model. One of the most difficult aspects of neural network modeling is finding the model hyperparameters that result in the best model. These hyperparameters are associated with the number of hidden layers, the number of hidden units, the activation function, the target function, and so on. You can tune the hyperparameters manually or you can use Model Studio's autotuning feature.

Network Learning Hyperparameters

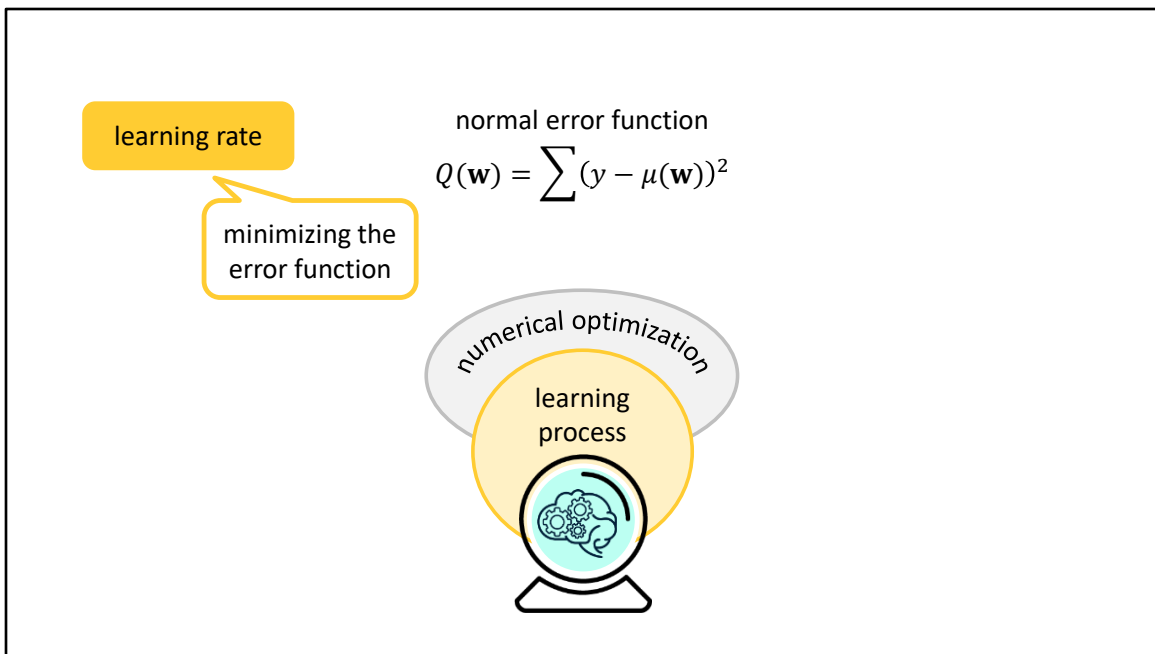


Copyright © SAS Institute Inc. All rights reserved.



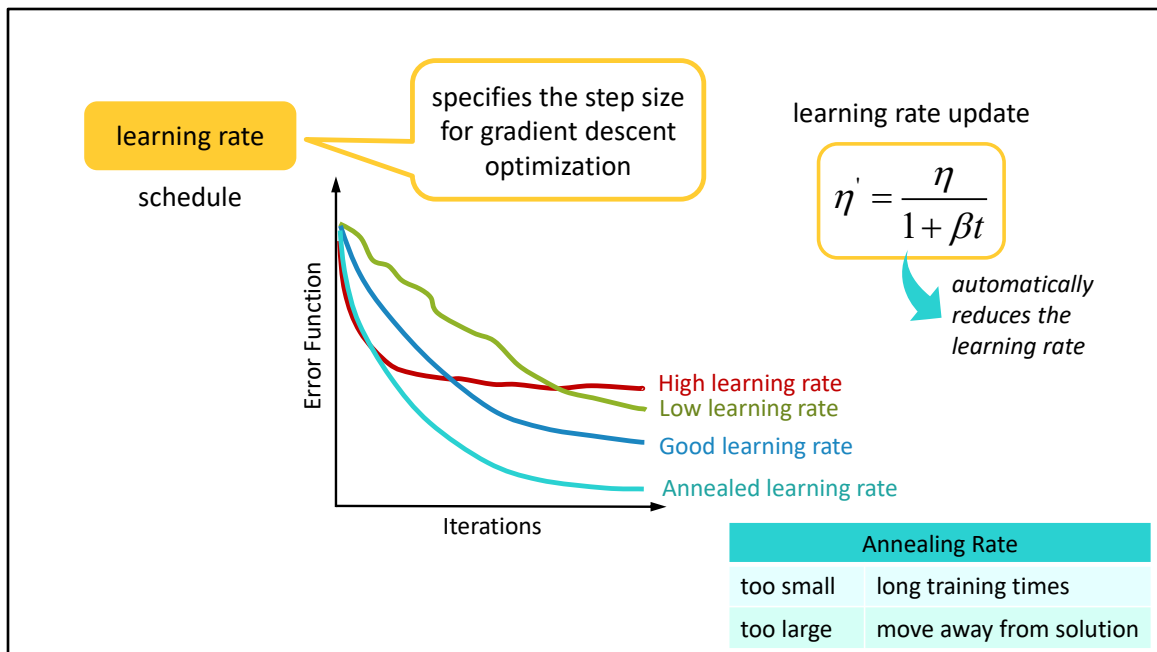
The learning process is critical during the building of a neural network. Remember that a neural network uses a numerical optimization method that you specify to estimate the weights that minimize the error function. This process is called learning.

A subset of important hyperparameters for neural networks directly affects the numerical optimization and thus the learning process. Therefore, these options affect the final performance of a neural network model. These hyperparameters include the learning rate, the annealing rate, the regularization terms L1 and L2, and the momentum.



The learning rate is defined in the context of optimization, and it is related to minimizing the error function (also known as the deviance function). For example, the default deviance measure that is used to fit an interval target is a normal distribution. This error function is the familiar ordinary least squares equation, shown here.

While minimizing the error function, we use for example, gradient descent or other variants of weight decay. In weight decay, the model parameters (that is, weights and biases in the network) are updated in a way that decreases the error function.



The learning rate is a training parameter that controls the size of weight and bias changes in learning the algorithm. Setting the learning rate too high yields great progress in decreasing the error function in the first few iterations, but at the risk of diverging later in the training process. On the other hand, setting the learning rate too small can result in very long training times. A "good" learning rate is a balance between the two and is considered to be problem specific.

A learning rate schedule is one approach used to optimize the learning process. The underlying idea is to train the model for a few iterations using a large learning rate. Then the learning rate is adjusted to a lower value, and the model is trained for a few more iterations. This process is repeated until convergence.

A learning rate schedule uses an annealing rate. Annealing is a way to automatically reduce the learning rate as stochastic gradient descent progresses, causing smaller steps as stochastic gradient descent approaches a solution. Effectively, it replaces the learning rate parameter, η , with η' , as shown in the equation. Here, t is the number of iterations that SGD has performed and is the annealing parameter.

Setting the annealing rate too small results in very long training times. Setting it too large can result in movement away from the desired solution (that is, divergence).

Important Hyperparameters for Neural Networks: Summary

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Modifying the Learning and Optimization Parameters

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Autotuning Options for Neural Networks in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



(Optional) Practice: Build a Neural Network Model Using Autotuning

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



4.4 Lesson Conclusion



Lesson Quiz

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.

